

CollabSphere WebApp

-Ho Chi Minh, 23 Jan 2026 -

Link GitHub

[Github Project CollabSphere](#) - GitHub - Phiadz/CNPM-friday

Link Youtube

[Youtube Project CollabSphere](#) - DEMO WEB COLLABSPHERE (Project-Based Learning Management System) - CNPM

Kết quả hoàn thành công việc

Tên	Role	Số Requirements	Tỉ Lệ % Đóng Góp
Lê Thiên Lộc	Backend Lead (Chính)	23	20.0%
Ngô Ngọc Phương Nghi	Backend Lead (Phụ)	22	19.1%
Trần Thuận Trí	Backend Developer	20	17.4%
Phạm Quốc Thiện	Frontend Developer 1	20	17.4%
Lý Huỳnh Bích Trâm	Frontend Developer 2	20	17.4%
Diệp Minh Thuận	Backend Developer	10	8.7%

Table of Contents

- [CollabSphere WebApp](#)
- [Acknowledgement](#)
- [I. Product introduction](#)
 - [1. Overview](#)
 - [1.1 Project Information](#)
 - [1.2 Project team](#)
 - [2. Product background](#)

- [3. Existing Systems](#)
 - [3.1. Communication Platforms](#)
 - [3.2. Project Management Tools](#)
 - [3.3. Collaborative Document & Whiteboard Tools](#)
 - [3.4. Learning Management Systems](#)
- [4. Business Opportunity](#)
- [5. Software Product Vision](#)
- [6. Project Scope & Limitations](#)
 - [6.1 Major Features](#)
 - [6.1.1 Major Features for Admin](#)
 - [6.1.2 Major Features for Head Department](#)
 - [6.1.3 Major Features for Staff](#)
 - [6.1.4 Major Features for Lecturer](#)
 - [6.1.5 Major Features for Student](#)
 - [6.2 Limitations & Exclusions](#)
- [II. Project Management Plan](#)
 - [1. Overview](#)
 - [1.1 Scope & Estimation](#)
 - [1.2 Project Objectives](#)
 - [1.2.1 Defect Distribution](#)
 - [1.3 Project Risks](#)
 - [2. Management Approach](#)
 - [2.1 Project Process](#)
 - [2.2 Quality Management](#)
 - [2.2.1 Quality Assurance Activities](#)
 - [2.3 Training Plan](#)
 - [3. Project Deliverables](#)
 - [4. Responsibility Assignments](#)
 - [5. Project Communications](#)
 - [6. Configuration Management](#)
 - [6.1 Document Management](#)
 - [6.2 Source Code Management](#)
 - [6.3 Tools & Infrastructure](#)
- [III. Software Requirement Specification](#)
 - [1. Product Overview](#)
 - [2. User Requirements](#)
 - [2.1 Actors](#)
 - [2.2 Use Cases](#)
 - [2.2.1 Diagram\(s\)](#)
 - [2.2.1.1 System Overall](#)

- [2.2.1.2 Administration & Academic Setup](#)
 - [2.2.1.3 Project Management & Teaching](#)
 - [2.2.1.4 Team Collaboration & Workspace](#)
 - [2.2.1.5 User's Authentication](#)
 - [2.2.2 Descriptions](#)
 - [2.2.2.1 System Overview](#)
 - [2.2.2.2 Administration & Academic Setup](#)
 - [2.2.2.3 Project Management & Teaching](#)
 - [2.2.2.4 Team Collaboration & Workspace](#)
 - [2.2.2.5 Authentication Services](#)
- [3. Functional Requirements](#)
 - [3.1 System Functional Overview](#)
 - [3.1.1 Screens Flow](#)
 - [3.1.2 Screen Descriptions](#)
 - [3.1.3 Screen Authorization](#)
 - [3.1.4 Non-Screen Functions](#)
 - [3.1.5 Entity Relationship Diagram](#)
 - [3.2. Authentication & User Management](#)
 - [3.3. Project & Team Administration](#)
 - [3.4. Real-time Collaboration Workspace](#)
 - [3.5. AI Assistant Integration](#)
 - [3.6. Evaluation & Assessment](#)
 - [3.7. Administrative & Academic Management](#)
- [4. Non-Functional Requirements](#)
 - [4.1. Performance](#)
 - [4.2. Security](#)
 - [4.3. Reliability & Availability](#)
 - [4.4. User Interface](#)
- [5. Requirement Appendix](#)
 - [5.1. Business Rules](#)
 - [5.2. Permission Matrix](#)
- [IV. Software Design Description](#)
 - [1. System Design](#)
 - [1.1 System Architecture](#)
 - [1.2 Package Diagram](#)
 - [2. Database Design](#)
 - [2.1 User & Authentication](#)
 - [2.2 Semester & Timeline](#)
 - [2.3 Profession Table](#)
 - [2.4 Specialty Table](#)

- [2.5 Subject Table](#)
- [2.6 Class Table](#)
- [2.7 Criteria Table](#)
- [2.8 Idea Table](#)
- [2.9 Topic Table](#)
- [2.10 Project Table](#)
- [2.11 Team Member Table](#)
- [2.12 Review Table](#)
- [2.13 Invitation Table](#)
- [2.14 Feedback Table](#)
- [2.15 Notification Table](#)
- [2.16 Message Table](#)
- [2.17 Grading / Rate Table](#)
- [2.18 Capstone Schedule Table](#)
- [2.19 Workspace Task & AI Assistant](#)
- [2.10 Syllabus Table](#)
- [3. Design Detail](#)
 - [3.1. Authentication](#)
 - [3.1.1 login](#)
 - [3.2. User Manager](#)
 - [3.2.1. Class Diagram](#)
 - [3.2.2 Sequence Diagram](#)
 - [3.3. Topic Management](#)
 - [3.3.1. Class Diagram](#)
 - [3.3.2. Sequence Diagram](#)
 - [3.4. Team Management](#)
 - [3.4.1. Class Diagram](#)
 - [3.4.2. Sequence Diagram](#)
 - [3.5. Review & Defense](#)
 - [3.5.1 Class Diagram](#)
 - [3.5.2. Sequence Diagram](#)
 - [3.6. Semester, Timeline, Criteria](#)
 - [3.6.1. Class Diagram](#)
 - [3.6.2. Sequence Diagram](#)
 - [3.7. Notification & Messaging](#)
 - [3.7.1. Class Diagram](#)
 - [3.7.2. Sequence Diagram](#)
 - [3.8. AI Assistant \(Google Gemini API\)](#)
 - [3.8.1. Class Diagram](#)
 - [3.8.2. Sequence Diagram](#)

- [V. Software Testing Documentation](#)
 - [1. Scope Of Testing](#)
 - [1.1 Student Role Test Features](#)
 - [1.2 Lecturer Role Test Features](#)
 - [1.3 Admin Role Test Features](#)
 - [1.4 Staff Role Test Features](#)
 - [1.5 Head Department Role Test Features](#)
 - [2. Testing Strategy and Levels](#)
 - [2.1 Unit Testing](#)
 - [2.2 Integration Testing](#)
 - [2.3 System Testing](#)
 - [2.4 Performance Testing](#)
 - [2.5 Supporting Tools](#)
 - [3. Test Plan](#)
 - [3.1 Human Resources](#)
 - [3.2 Test environment and tool](#)
 - [4. Test Reports](#)
 - [4.1 Test Cases](#)
 - [4.2 Test Results Report](#)
 - [4.3 Final Evaluation](#)
- [VI. Release Package & User Guides](#)
 - [1.Deliverable Package](#)
 - [2.Installation Guides](#)
 - [2.1 Prerequisites](#)
 - [2.1.1 System Requirements](#)
 - [2.2 Installation Instructions](#)
 - [2.2.1 Backend](#)
 - [2.2.2 Frontend](#)
 - [3.User Manual](#)
 - [3.1 Roles and Features](#)
 - [3.2 Workflow Examples](#)
 - [3.2.1 Login](#)
 - [3.2.2 Access Dashboard](#)
 - [3.2.3 Team Management](#)
 - [3.2.4 Invite Member](#)
 - [3.2.5 Real-time Workspace \(Main Screen\)](#)
 - [3.2.6 Kanban Board Detail](#)
 - [3.2.7 Whiteboard Canvas](#)
 - [3.2.8 Video Meeting Room](#)
 - [3.2.9 Submission Portal](#)

- [3.2.10 Peer Review Form](#)
- [3.2.11 Lecturer Dashboard](#)
- [3.2.12 Topic Management](#)
- [3.2.13 Class Progress Monitoring](#)
- [3.2.14 Grading & Feedback Interface](#)
- [3.2.15 Staff - Semester Management](#)
- [3.2.16 Staff - User Imports](#)
- [3.2.17 Head of Dept - Statistics Dashboard](#)
- [3.2.18 Head of Dept - Topic Bank Approval](#)
- [3.2.19 System Configuration](#)

Acknowledgement

We would like to express our deepest gratitude to all those who have contributed to the successful completion of this project, CollabSphere (COSRE) – A System to Support Learning Using Project-Based Learning Method.

First and foremost, we extend our sincere appreciation to our project supervisor, Nguyễn Văn Chiến, for his invaluable guidance, insightful feedback, His expertise have been instrumental in shaping the direction and quality of this work.

We would also like to thank our friends and family for their unwavering support, understanding, and motivation throughout this journey. Their encouragement helped us stay focused and resilient during demanding periods.

This project would not have been possible without the collective effort and support of everyone mentioned above.

Definition and Acronyms

Acronym	Definition
PBL	Project-Based Learning. A teaching method in which students learn by actively engaging in real-world and personally meaningful projects ¹ .
LMS	Learning Management System. A software application for the administration, documentation, tracking, reporting, and delivery of educational courses or training programs ² .

RBAC	Role-Based Access Control. A mechanism that restricts system access to authorized users based on their assigned roles (e.g., Admin, Staff, Lecturer, Student) ³ .
JWT	JSON Web Token. An open standard used for securely transmitting information between parties as a JSON object, utilized here for user authentication and session management.
SPA	Single Page Application. A web application architecture (using ReactJS) that interacts with the user by dynamically rewriting the current web page with new data from the web server.
API	Application Programming Interface. A set of protocols and definitions that allows the Frontend (React) to communicate and exchange data with the Backend (FastAPI) ⁴ .
ORM	Object-Relational Mapping. A programming technique used to convert data between the object-oriented system (Python) and the relational database (PostgreSQL), specifically using SQLAlchemy.
LLM	Large Language Model. An advanced AI algorithm (such as Google Gemini) integrated into the system to provide intelligent suggestions, chatbot support, and data analysis ⁵ .
WebRTC	Web Real-Time Communication. An open-source project providing web browsers and mobile applications with real-time communication capabilities via simple APIs, used for video conferencing ⁶ .
WebSocket	WebSocket Protocol. A computer communications protocol providing full-duplex communication channels over a single TCP connection, implemented via Socket.IO for real-time features like chat and whiteboard ⁷ .
UUID	Universally Unique Identifier. A 128-bit label used for information in computer systems, utilized as the Primary Key for users to ensure security and scalability.
ERD	Entity Relationship Diagram. A flowchart that illustrates how "entities" such as people, objects, or concepts relate to each other within a system's database.
HoD	Head of Department. A specific user role within the system responsible for approving project topics and monitoring department-level statistics ⁸ .
Kanban	Kanban. A visual workflow management method implemented in the system as a board with columns (To Do, Doing, Done) to manage team tasks effectively ⁹ .

Sprint	Sprint. A set period of time during which specific tasks must be completed and made ready for review, a core component of the Agile methodology used in the project ¹⁰ .
Rubric	Rubric. A scoring guide used to evaluate the quality of students' constructed responses, applicable for project grading and peer reviews ¹¹ .
CRUD	Create, Read, Update, Delete. The four basic operations of persistent storage that represent the core functionalities of the system's API.
UML	Unified Modeling Language. A standardized modeling language used to visualize the design of a system, including Use Case, Activity, and Sequence diagrams ¹² .
3NF	Third Normal Form. A database schema design standard used to minimize data redundancy and ensure data integrity.

I. Product introduction

1. Overview

1.1 Project Information

Project name: CollabSphere - System to support learning using project-based learning method

Group name: CollabSphere

Software type: Web application

1.2 Project team

Full name	Role	Email
Ts. Nguyễn Văn Chiến	Supervisor	chiennv@ut.edu.vn
Lê Thiên Lộc	Leader	loclt0024@ut.edu.vn
Ngô Ngọc Phương Nghi	Member	nghinnp2852@ut.edu.vn
Lý Huỳnh Bích Trâm	Member	tramlhb6968@ut.edu.vn

Phạm Quốc Thiện	Member	thienpq9681@ut.edu.vn
Diệp Minh Thuận	Member	thuandm0458@ut.edu.vn
Trần Thuận Trí	Member	tritt8855@ut.edu.vn

2. Product background

Project-Based Learning (PBL) has established itself as a highly effective educational methodology fostering practical skills, critical thinking, and teamwork by engaging students in real-world projects. However, the execution and management of PBL in academic settings are hampered by significant logistical challenges.

Currently, students and lecturers are forced to use a fragmented ecosystem of disparate tools for different needs like separate platforms for communication (e.g., chat apps), video conferencing, document sharing, diagramming and task/progress tracking. This constant switching between applications disrupts workflow, reduces efficiency, and fragments the project's context. Furthermore, existing solutions often lack deep, real-time collaborative features tailored for the iterative, creative process of PBL and fail to provide a unified system for transparent progress tracking and fair assessment of individual contributions within a team.

This context presents a clear opportunity to build a dedicated, integrated platform that streamlines the entire PBL lifecycle, from project setup to final evaluation, within a single, cohesive digital workspace.

3. Existing Systems

3.1. Communication Platforms

Description: Communication platforms such as Zoom, Microsoft Teams, and Google Meet are widely used in both educational and professional settings to facilitate remote interaction. They provide essential real-time communication features, including video and audio conferencing, instant messaging, and screen sharing.

Functionality: Video conferencing, chat, screen sharing.

Limitations: Not designed for academic use; lack features for instructor evaluation, peer review, or integration with real-time collaborative tools like whiteboards or document editors.

3.2. Project Management Tools

Description:

Project management tools like Trello, Asana, and Jira help teams organize tasks, track progress, and manage workflows through visual boards, timelines, and assignment systems.

Functionality: Task boards, milestone tracking, assignment management

Limitations: Not designed for academic use; lack features for instructor evaluation, peer review, or integration with real-time collaborative tools like whiteboards or document editors.

3.3. Collaborative Document & Whiteboard Tools

Description:

Tools such as Google Docs, Miro, and FigJam enable real-time co-editing, brainstorming, and visual collaboration. They allow multiple users to simultaneously work on documents, diagrams, or mind maps, making them valuable for creative and planning phases of projects.

Functionality: Real-time editing, brainstorming, diagramming.

Limitations: Isolated from communication and project tracking systems. Do not support structured evaluation, progress analytics, or role-based access for academic roles (lecturers, students, admins).

3.4. Learning Management Systems

Description:

Learning Management Systems like Moodle, Canvas, and Blackboard are foundational in digital education for course administration, content distribution, assignment submission, and grade management.

Functionality: Course management, resource sharing, grading.

Limitations: Limited real-time collaboration features; often lack integrated video conferencing, interactive whiteboards, or AI-assisted project planning. Not optimized for agile team-based project development.

3.5. File Sharing & Resource Repositories

Description:

Cloud storage services such as Google Drive, OneDrive, and Dropbox offer centralized file management, version control, and sharing capabilities. They are commonly used to store and distribute project resources, documents, and media.

Functionality: Centralized file storage and sharing.

Limitations: No built-in project tracking, communication, or evaluation capabilities. Often used as a supplementary tool rather than a core collaboration environment.

4. Business Opportunity

CollabSphere addresses a clear and growing need in the educational technology sector by offering a unified, purpose-built platform for project-based learning (PBL). The fragmented use of generic tools creates significant inefficiencies, presenting a strong market opportunity for an integrated academic collaboration system.

5. Software Product Vision

Our vision is to become the leading, all-in-one collaborative platform where academic theory meets practical execution—seamlessly integrating communication, project management, real-time co-creation, and insightful evaluation into a single, intuitive environment.

We aim to eliminate the friction of using multiple disjointed tools, empowering students and educators to focus on what truly matters: innovation, deep learning, and meaningful collaboration. CollabSphere will be the central nervous system for academic projects, fostering a culture of transparency, accountability, and shared achievement.

6. Project Scope & Limitations

6.1 Major Features

The system will include the following core features, organized by functional modules:

6.1.1 Major Features for Admin

FE-01: Login/Logout

FE-02: View all accounts (Head Department, Staff, Lecturer, Student)

FE-03: Deactivate all user accounts

FE-04: View all system reports from users (via email)FE-05: Dashboard

6.1.2 Major Features for Head Department

FE-01: Login/Logout

FE-02: View list and detail of all classes

FE-03: View list and detail of subject & syllabus

FE-04: View list and detail of all projects (pending, approved, denied)

FE-05: Approve/deny pending projects

FE-06: Update approved projects

FE-07: Assign projects from approved projects list for all classes

6.1.3 Major Features for Staff

FE-01: Login/Logout

FE-02: Import files to automatically create and manage subjects, syllabus, classes, lecturer/student accounts

FE-03: View list and detail of subjects and syllabus, classes, lecturer/student accounts

FE-04: Assign lecturer and students to classes

6.1.4 Major Features for Lecturer

FE-01: Login/Logout

FE-02: View and manage own projects (create, submit for approval, assign to classes)

FE-03: Use AI to generate project milestones

FE-04: View and manage assigned classes (including list of students, projects, teams)

FE-05: Create and manage teams in assigned classes

FE-06: Monitor team progress and individual contributions

FE-07: Use AI for analysis and recommendations on team progress and individual contributions

FE-08: Manage team milestones and create milestones questions

FE-09: View milestones answers and checkpoint submissions

FE-10: View workspace of teams

FE-11: Manage resources for classes, teams, milestones, checkpoints

FE-12: Evaluate and give feedback for teams and individual members (at project end, for milestones, for checkpoints)

FE-13: View evaluations and feedback from peers (for team members)

FE-14: Communicate via chat, call meetings, schedule meetings

FE-15: Use real-time collaboration tools (whiteboard, text editor)

FE-16: Use AI chatbot

6.1.5 Major Features for Student

FE-01: Login/Logout

FE-02: View assigned classes and team details

FE-03: View subject/syllabus, team milestones and checkpoints

FE-04: Monitor team progress and individual contributions

FE-05: Use AI for analysis and recommendations on team progress and individual contributions

FE-06: Mark done team milestones and checkpoints

FE-07: View and answer milestones questions

FE-08: Create, manage, and submit team checkpoints

FE-09: View and manage workspace (cards, tasks, subtasks)

FE-10: Manage resources for classes, teams, milestones, checkpoints

FE-11: Evaluate and give feedback for other team members (at project end, for milestones answers)

FE-12: Communicate via chat, call meetings, schedule meetings

FE-13: View evaluations and feedback from lecturer and peers

FE-14: Use real-time collaboration tools

FE-15: Use AI chatbot

6.2 Limitations & Exclusions

LI-1: The system does not include built-in video conferencing but integrates with WebRTC for real-time communication, which may require additional configuration and is dependent on third-party services.

LI-2: The AI features (chatbot, milestone generation, analysis) are provided as supportive tools and may not cover all possible scenarios or provide accurate results in every case.

LI-3: The system is designed for use within a single institution and does not support multi-institution collaboration out of the box.

LI-4: The system does not include advanced reporting and analytics features beyond the provided dashboards and progress tracking.

LI-5: The system does not support offline mode; all real-time collaboration features require an active internet connection.

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

#	WBS Item	Complexity	Est. Effort (man-days)
1	Initiating		20
1.1	Define project scope	Medium	12
1.2	Collect and analyze requirements	Medium	8
2	Planning		14
2.1	Create kick-off meeting	Medium	7
2.2	Create project management plan document	Medium	7
3	Executing		
3.1	Analysis		18
3.1.1	Analyze functional requirements	Medium	8
3.1.2	Feasibility analysis (technical & schedule)	Complex	10
3.2	Design		12
3.2.1	Design conceptual ERD	Medium	4
3.2.2	Design system & code architecture	Medium	4
3.2.3	Design web application (UI & flow)	Medium	4
3.3	Implementation		
3.3.1	Authentication & Authorization		6
3.3.1.1	Login / Logout	Medium	2
3.3.1.2	Role-based access control	Medium	2

3.3.1.3	Token validation & refresh	Medium	2
3.3.2	Account Management		10
3.3.2.1	Manage admin, staff, lecturer, student accounts	Medium	4
3.3.2.2	Import accounts from file	Medium	3
3.3.2.3	Assign roles and classes	Medium	3
3.3.3	Subject & Syllabus Management		8
3.3.3.1	Import subject and syllabus data	Medium	4
3.3.3.2	Manage subject & syllabus information	Medium	4
3.3.4	Class Management		8
3.3.4.1	Create and manage classes	Medium	4
3.3.4.2	Assign lecturers and students to classes	Medium	4
3.3.5	Project Management		14
3.3.5.1	Create project based on syllabus	Medium	4
3.3.5.2	Project approval workflow	Medium	4
3.3.5.3	Assign project to classes	Medium	3
3.3.5.4	Manage project milestones	Medium	3
3.3.6	Team & Workspace Management		16
3.3.6.1	Create and manage teams	Simple	3
3.3.6.2	Task board (cards, tasks, subtasks)	Medium	5
3.3.6.3	Sprint & progress tracking	Medium	4
3.3.6.4	Contribution calculation	Medium	4
3.3.7	Realtime Communication		12

3.3.7.1	Team chat system	Complex	6
3.3.7.2	Video & audio meetings	Complex	6
3.3.8	Realtime Collaboration Tools		12
3.3.8.1	Whiteboard collaboration	Complex	6
3.3.8.2	Realtime document editing	Complex	6
3.3.9	Evaluation & Feedback System		10
3.3.9.1	Lecturer evaluation & feedback	Medium	4
3.3.9.2	Peer evaluation	Medium	3
3.3.9.3	Feedback visualization	Medium	3
3.3.10	AI Assistance		8
3.3.10.1	AI project information generation	Medium	4
3.3.10.2	AI chatbot support	Medium	4
3.4	Testing		24
3.4.1	Unit testing	Complex	8
3.4.2	Integration testing	Complex	8
3.4.3	System testing	Complex	8
3.5	Monitoring and Controlling		24
3.5.1	Control project progress	Complex	12
3.5.2	Track performance and quality	Complex	12
3.6	Closing		20
3.6.1	Final report and documentation	Simple	20

1.2 Project Objectives

Project Constraints

- **Timeline:** The project must be finished before **30 April 2025**
- **Allocated Effort (Man-days): 230**

1.2.1 Defect Distribution

#	Testing Stage	Test Coverage	No. of Defects	% of Defect	Notes
1	Reviewing	Backend: Code review (Authentication, RBAC, Core APIs) Frontend: Code review (React components, state management)	< 30	5%	Focus on security, performance, and maintainability
2	Unit Test	Backend: Business logic, API validation Frontend: Component testing, form validation	< 20	3%	Ensure correctness of core logic and data handling
3	Integration Test	Backend: API integration, database consistency Frontend: API binding, UI behavior	< 10	2%	Verify interaction between frontend, backend, and database
4	System Test	Backend: Full system testing with database (PostgreSQL, Redis) Frontend: End-to-end user flows	< 5	1%	Ensure smooth UI–Backend communication and system stability
5	Acceptance Test	System: Full project workflow	< 5	1%	Ensure system meets functional and academic requirements

1.3 Project Risks

#	Risk Description	Impact	Possibility	Response Plans
1	Incomplete or unclear requirements for project-based learning workflows	System may not fully support academic processes, leading to rework	Medium	Conduct requirement reviews with all team members and freeze scope early
2	Integration issues between frontend, backend, and real-time modules	Delays and unstable system behavior	Medium	Define API contracts early and perform continuous integration testing
3	Real-time communication features (chat, video meetings, whiteboard) malfunction	Collaboration experience is degraded	High	Implement simplified real-time features and test incrementally
4	Workload imbalance among team members	Reduced productivity and missed deadlines	Medium	Clearly assign roles and monitor progress through sprint reviews
5	Limited experience with cloud deployment and real-time technologies	Deployment delays or system instability	Medium	Set up deployment environment early and allocate buffer time
6	Data security and access control issues	Unauthorized access or data leakage	Low	Apply role-based access control and validate authentication logic
7	Time constraints due to fixed academic schedule	Incomplete feature implementation	High	Prioritize core features and defer non-essential functionalities

2. Management Approach

2.1 Project Process

The CollabSphere (COSRE) project follows an **Iterative and Incremental Development Process** based on Agile principles. The system is developed through multiple iterations (sprints), where each sprint delivers a functional and testable increment of the system.

The project execution is divided into clearly defined phases: analysis, design, implementation, testing, and deployment. Sprint-based development allows early validation of features, timely detection of risks, and continuous integration between system components.

This process is suitable for COSRE due to the system’s complexity, real-time collaboration requirements, and limited project duration.

2.2 Quality Management

Quality assurance activities are conducted throughout the project lifecycle to ensure system correctness, stability, and maintainability.

2.2.1 Quality Assurance Activities

Activity	Description
Code Review	Review backend and frontend source code to ensure coding standards and security
Unit Testing	Test individual backend services and frontend components
Integration Testing	Verify interactions between frontend, backend, database, and real-time modules
System Testing	Validate complete workflows against requirements
Bug Tracking	Log, track, and fix defects using version control issues

Quality issues are addressed early to reduce rework in later stages.

2.3 Training Plan

To ensure effective collaboration and technical consistency, team members participate in short training sessions during the initial phase of the project.

Training Content	Participants	Time	Duration	Requirements
Project overview & workflow	All members	Week 1	1 day	Mandatory
Backend technologies (Python API, PostgreSQL)	Backend members	Week 1	3 days	Mandatory
Frontend technologies (ReactJS)	Frontend members	Week 1	3 days	Mandatory
Git & collaboration tools	All members	Week 1	1 day	Mandatory
Cloud & deployment basics	DevOps member	Week 2	2 days	Mandatory

3. Project Deliverables

#	Deliverable	Due Date	Notes
1	Sprint 1	Week 1	Collect and analyze requirements Define project scope Project planning and training
2	Sprint 2	Week 2	Design database schema (ERD)Design system architecture Set up backend project structure
3	Sprint 3	Week 3	- Implement authentication and authorization module - Implement account management module
4	Sprint 4	Week 4	- Implement subject, syllabus, and class management modules - Implement project and team management

5	Sprint 5	Week 5	· Implement team workspace and task board · Implement progress and contribution tracking
6	Sprint 6	Week 6	· Implement real-time communication (chat, notifications) · Implement evaluation and feedback system
7	Sprint 7	Week 7	· System testing and bug fixing · Deployment and release · Prepare final report and presentation

4. Responsibility Assignments

(D: Do, R: Review, S: Support, I: Inform, blank: Omitted)

Responsibility	Leader	BE 1	BE 2	BE 3	BE 4	FE 1	FE 2
Project planning & tracking	D	I	I	I	I	I	I
Requirement analysis	D	S	S	S	S	S	S
Prepare Software Requirement Specification (SRS)	R	D	S	S	I	S	I
Prepare Software Design Document (SDD)	R	S	D	S	I	I	D
Database design (ERD)	R	D	S	I	I	I	I
Backend core implementation	I	D	I	I	I	I	I
Realtime & communication implementation	I	S	D	I	I	I	I
Frontend core implementation	S	I	I	D	S	I	I
Frontend collaboration tools	I	I	I	I	I	D	S
Integration & API coordination	S	I	I	S	I	S	D

Testing (unit, integration, system)	D	S	S	S	S	D	S
Deployment & configuration	R	S	S	S	S	S	D
Prepare user & installation guides	S	S	I	I	D	I	I
Prepare final report	D	S	S	S	S	S	S
Final presentation & demo	D	S	S	S	S	S	S

5. Project Communications

Communication Item	Who / Target	Purpose	When / Frequency	Type, Tool, Method(s)
Team weekly meeting	All team members	Review sprint plan, task progress, completed work, and upcoming activities	Once per week (end of sprint)	Online meeting (Google Meet)
Sprint planning meeting	All team members	Plan sprint tasks, assign responsibilities, and estimate workload	At the beginning of each sprint	Online meeting (Google Meet)
Daily meeting	Team members	Report daily progress, discuss blockers, and coordinate implementation	Daily	Online meeting (Google Meet / Chat)
Weekly report	Project manager → Supervisor	Summarize project progress, completed tasks, issues, and next-week plan	Once per week	Document submission / Email
Technical discussion	Backend & Frontend members	Resolve technical issues related to APIs, database, and integration	As needed	Online discussion (Chat / Google Meet)
Unscheduled meeting	All team members	Handle critical issues that require immediate resolution	When necessary	Online meeting (Google Meet)

Final presentation meeting	All team members & Supervisor	Present final system, demonstrate features, and receive feedback	End of project	Online presentation (Google Meet)
----------------------------	-------------------------------	--	----------------	-----------------------------------

6. Configuration Management

6.1 Document Management

All project-related documents are managed in a centralized and consistent manner to ensure version control and accessibility among team members.

- **Document tools:**

Microsoft Word, Microsoft Excel

- **File management and sharing:**

Google Drive is used to store, share, and manage all project documents, including requirements, design documents, reports, and presentation slides.

Access permissions are controlled to avoid unauthorized modification.

6.2 Source Code Management

The source code of the CollabSphere (COSRE) system is managed using a distributed version control system.

- **Version control tool:**

Git

- **Source code repository:**

GitHub

- **Branching strategy:**

Feature branches are created for development tasks and merged into the main branch after review to ensure code stability and traceability.

6.3 Tools & Infrastructure

Category	Tools / Infrastructure
Frontend	ReactJS, Vite, Ant Design / MUI

Backend	Python FastAPI
Database	PostgreSQL
Cache / Message Broker	Redis
Realtime Communication	Socket.IO (Chat, Whiteboard)
Video / Audio Call	PeerJS or LiveKit
File Storage	Cloudinary (API-based upload for images and files)
AI Integration	Google Gemini API
IDE / Editors	Visual Studio Code
Diagramming	Flowchart Maker & Online Diagram Software
Documentation	Microsoft Word, Excel, Google Docs
Version Control	GitHub
Deployment & DevOps	Docker, Docker Compose
Project Management	Google Sheets

III. Software Requirement Specification

1. Product Overview

CollabSphere (COSRE) is an "All-in-one" integrated web application system designed to support the Project-Based Learning method. The system addresses the issue of tool fragmentation by centralizing project management, real-time communication (chat, video call), collaboration (whiteboard), and evaluation (grading/peer review) onto a single platform.

Key Features:

- Project & Team Management.
- Real-time Workspace (Chat, Meeting, Whiteboard).

- AI Assistant.
- Evaluation & Grading.
- School's management

2. User Requirements

2.1 Actors

The system serves the following user groups with specific characteristics:

#	Actor	Description
1	admin	System managers responsible for managing user accounts, configuring semesters, and technical parameters.
3	Staff	School's staff team who jobs is to create, import, manage syllabus, subjects, classes. They can assign lecture and student to their classes
2	HoD	Approvers of subject syllabus or general quality supervisors.
3	Lecturer	Supervisors who create project topics, track team progress, grade assignments, and provide feedback.
4	Student	Participants in the learning process who need to find teams, execute projects, collaborate with other members, and submit deliverables. They require a user-friendly interface and fast interaction.
5	Leader	A role in the student's team that can use some features some the Lecture actor. Requirement of this actor is exactly like the student actor
6	User	A generalized actor representing any individual accessing the CollabSphere system before their specific role is identified. This actor serves as the parent entity for all other actors, responsible for common security interactions such as authentication, termination of session, and password recovery.

2.2 Use Cases

2.2.1 Diagram(s)

2.2.1.1 System Overall

2.2.1.2 Administration & Academic Setup

2.2.1.3 Project Management & Teaching

2.2.1.4 Team Collaboration & Workspace

2.2.1.5 User’s Authentication

2.2.2 Descriptions

2.2.2.1 System Overview

ID	Use Case	Actors	Use Case Description
OVR-01	Do Authentication	User	Handle login, logout, and security verification (Base for all actors).
OVR-02	Manage System & Accounts	Admin, Staff	Configure system parameters and manage user lifecycles.
OVR-03	Manage Academic Data	Staff	Handle core academic entities like Subjects, Syllabi, and Classes.
OVR-04	Manage Projects	Lecturer, HoD	Oversee the lifecycle of project topics from creation to approval.

OVR-05	Manage Teams & Classes	Lecturer, Staff	Organize students into classes and project teams.
OVR-06	Execute Project & Collaborate	Student	Perform project tasks and collaborate using real-time tools.
OVR-07	Evaluate & Grade	Lecturer	Assess student performance and project outcomes.

2.2.2.2 Administration & Academic Setup

ID	Use Case	Actors	Use Case Description
ADM-01	Manage Users	Admin	View lists of Staff, Head Dept, Lecturers, Students; deactivate accounts if necessary.
ADM-02	View System Reports	Admin	Monitor overall system statistics and health dashboard.
ACA-01	Manage Subjects	Staff	Import subject lists from files or remove existing subjects.
ACA-02	Manage Syllabus	Staff	Import syllabus details for subjects or remove outdated syllabi.
ACA-03	Manage Classes	Staff	Create classes, import student lists, assign lecturers, or remove classes.
ACA-04	Import Accounts	Staff	Bulk create Lecturer and Student accounts via file import.
ACA-05	Manage Pending Projects	Head of Dept	Review project topics submitted by Lecturers (Approve or Deny).
ACA-06	Assign Projects to Classes	Head of Dept	Distribute approved project topics to specific academic classes.

2.2.2.3 Project Management & Teaching

ID	Use Case	Actors	Use Case Description
PRJ-01	Create Project	Lecturer	Define project specifications including objectives and milestones.
PRJ-02	Use AI to Generate Milestones	Lecturer	Leverage AI to automatically suggest project milestones based on objectives.
PRJ-03	Submit Project for Approval	Lecturer	Send the created project topic to the Head of Department for review.
PRJ-04	Create Teams	Lecturer	Form student teams within the assigned classes.
PRJ-05	Assign Project to Team	Lecturer	Allocate specific approved projects to student teams.
PRJ-06	Monitor Progress	Lecturer	Track team performance via dashboards; use AI to analyze individual contributions.
PRJ-07	Evaluate & Feedback	Lecturer	Grade checkpoint submissions and the final project outcome.
PRJ-08	Update Approved Projects	Head of Dept	Modify details of projects that have already been approved.

2.2.2.4 Team Collaboration & Workspace

ID	Use Case	Actors	Use Case Description
COL-01	Manage Workspace	Student	Organize teamwork by creating tasks, updating status, and managing sprints.
COL-02	Real-time Collaboration	Student	Interact synchronously via Whiteboard, Collaborative Text Editor, Group Chat, or Video Meeting.

COL-03	Use AI Assistant	Student	Chat with the AI bot for brainstorming, technical guidance, or Q&A.
EXE-01	Project Execution	Student	Perform project activities including submitting milestones and checkpoints.
EXE-02	Peer Review	Student	Evaluate and provide feedback on other team members' contributions.
EXE-03	Mark Milestone Done	Team Leader	Officially declare a project milestone as completed (Leader privilege).
tch-01	View Team Workspace	Lecturer	Access and observe the working environment/progress of student teams.
tch-02	Join Team Meeting	Lecturer	Participate in student team video meetings for guidance or review.
tch-03	Upload Resources	Lecturer	Share learning materials or documents with the team.

2.2.2.5 Authentication Services

ID	Use Case	Actors	Use Case Description
AUTH-01	Log in	User	Authenticate users into the system using username/password or linked accounts.
AUTH-02	Register	User	New users create a personal account to access the system.
AUTH-03	Log out	User	Terminate the current session securely.
AUTH-04	Forgot Password	User	Recover account access via Email or Phone Number verification.
AUTH-05	Use Third-Party Authentication	User	Log in quickly using external providers (Google, Microsoft, etc.).

AUTH-06	Use Dashboard	User	Access the main landing page containing summarized information after logging in.
AUTH-07	Change User's Name	User	Update the display name in the user profile.
AUTH-08	Add Contact Info	User	Add or update Email and Phone Number for account verification and notifications.

3. Functional Requirements

This section describes the detailed functions of the system.

3.1 System Functional Overview

3.1.1 Screens Flow

Student Screen Flow

Lecture Screen FLOW

Staff Screen Flow

Head of Department Screen Flow

System Admin Screen Flow

3.1.2 Screen Descriptions

This section describes the key user interfaces of the CollabSphere system.

[Group 1: Authentication & Common]

- **SCR-01: Landing Page & Login:** Main entry point. Contains "Sign in with Google" (OAuth2), standard Login form, and "Forgot Password" link.
- **SCR-02: User Profile & Settings:** Allows users to view personal info, update avatar, change password, and view assigned roles.
- **SCR-03: Notification Center:** A dropdown or dedicated page listing system alerts (New task, Deadline warning, New message).

[Group 2: Student Workspace]

- **SCR-04: Student Dashboard:** Displays active projects, upcoming deadlines (Timeline view), and quick access to recent files.
- **SCR-05: Team Management:**
 - o *Leader View:* Buttons to "Invite Member", "Kick Member", "Assign Role".
 - o *Member View:* List of teammates and contact info.
- **SCR-06: Real-time Workspace (Main Screen):**
 - o *Integrated View:* Split screen showing Chat (Right), Whiteboard/Task (Center), and Video Call Header (Top).
- **SCR-07: Kanban Board Detail:** Columns (To Do, Doing, Done), Task Cards with tags (Priority, Due Date), and "Add Task" modal.
- **SCR-08: Whiteboard Canvas:** Full-screen mode with toolbar (Pen, Shape, Note, Eraser, Undo/Redo) and cursor presence of other members.
- **SCR-09: Video Meeting Room:** Grid view of webcams, toolbar (Mute, Stop Video, Share Screen, Chat, Leave).
- **SCR-10: Submission Portal:** Form to upload zip files/documents or paste GitHub links for project milestones.
- **SCR-11: Peer Review Form:** Matrix for rating teammates (Contribution, Attitude, Skill) with sliding scales and comment text areas.

- **SCR-12: Project Browser / Join Team:** An intermediate screen allowing students to browse available topics or teams and request to join before accessing the main Student Dashboard (SCR-04)

[Group 3: Lecturer Management]

- **SCR-13: Lecturer Dashboard:** Overview of all classes, total number of teams, and pending grading requests.
- **SCR-14: Topic Management:** List of created topics with buttons to "Create New Topic", "Edit", "Delete", and "Approve Student Proposed Topic".
- **SCR-15: Class Progress Monitoring:** A table view showing all teams in a class with columns for "Last Activity", "Completed Tasks", "Submission Status".
- **SCR-16: Grading & Feedback Interface:** Split screen: Left side shows student submission (PDF/Code viewer), Right side shows Rubric scoring and Feedback box.

[Group 4: Head of Department & Staff]

- **SCR-17: Staff - Semester Management:** Interface to create new semesters, set start/end dates, and configure milestones.
- **SCR-18: Staff - User Import:** Interface to upload Excel/CSV files to create accounts for Students/Lecturers in bulk.
- **SCR-19: Head of Dept - Statistics Dashboard:** Charts showing pass/fail rates, project quality distribution, and resource usage.
- **SCR-20: Head of Dept - Topic Bank Approval:** List of topics submitted by Lecturers waiting for Department approval.
- **SCR-21: Subject & Class Management:** Interface for Staff to view the subject list, create new academic classes, and assign lecturers

[Group 5: System Admin]

- **SCR-22: System Configuration:** Settings for AI API Keys (AWS Bedrock), Email Server (SMTP), and Cloudinary integration.

· **SCR-23: Admin Dashboard & User List:** A central hub for Admins to manage the user registry (CRUD operations) and toggle account status (Activate/Deactivate), distinct from the bulk import function in SCR-17

3.1.3 Screen Authorization

This section shows who can see each screen

Screen ID	Screen Name	Student	Lecturer	Head of Dept	Staff	Admin
SCR-01	Login Page	✓	✓	✓	✓	✓
SCR-02	User Profile	✓	✓	✓	✓	✓
SCR-04	Student Dashboard	✓	✗	✗	✗	✗
SCR-06	Workspace (Chat/Board)	✓	✓ (View)	✗	✗	✗
SCR-12	Lecturer Dashboard	✗	✓	✗	✗	✗
SCR-13	Topic Management	✗	✓	✓ (View)	✗	✗
SCR-15	Grading Interface	✗	✓	✓ (View)	✗	✗
SCR-16	Semester Config	✗	✗	✗	✓	✗
SCR-17	User Import	✗	✗	✗	✓	✓
SCR-18	Statistics Dashboard	✗	✗	✓	✗	✗
SCR-20	System Settings	✗	✗	✗	✗	✓

(Symbol Key: ✓ = Allowed, ✗ = Denied)

3.1.4 Non-Screen Functions

This section describes system functionalities that operate in the background or via API integrations without direct user interface interaction.

NSF-01: Automated Deadline Notification (Cron Job)

- **Description:** The system automatically scans for upcoming deadlines (24 hours and 1 hour remaining) every 30 minutes.
- **Action:** Triggers an email via SMTP and a push notification to all team members involved in the task or submission.

NSF-02: AI Processing Service (AWS Bedrock Integration)

- **Description:** The middleware handles communication between the Chatbot UI and the AWS Bedrock LLM.
- **Process:** Receives user prompt -> Sanitizes input -> Constructs context (Subject, Project info) -> Sends to AWS -> Formats response -> Returns to UI.

NSF-03: Video Recording Processor

- **Description:** Handles the processing of recorded video meetings.
- **Process:** After a meeting ends, the temporary video stream is merged, compressed (FFmpeg), and uploaded to Cloud Storage (Cloudinary). A link is generated and saved to the Meeting History.

NSF-04: Real-time Socket Server (WebSocket)

- **Description:** Manages persistent connections for instant data synchronization.
- **Scope:** Syncs Whiteboard drawing coordinates, Chat messages, and Online status presence without requiring page reloads.

NSF-05: Semester Archiving

- **Description:** An automated job that runs at the configured "Semester End Date".
- **Action:** Changes the status of all projects in that semester to "Archived", makes data Read-Only, and frees up active server resources.

NSF-06: Plagiarism Checking (Optional/Placeholder)

- **Description:** Basic comparison of text submissions against the internal Topic Bank database to detect duplicate project proposals.

3.1.5 Entity Relationship Diagram

The database design of CollabSphere reflects the complexity of a multi-module system, ensuring data integrity, scalability, and performance for real-time interactions,. This is a physical ERD.

The data model is normalized to Third Normal Form (3NF) to minimize redundancy and is organized into five main functional clusters:

1. **Identity Management:** Handles Users, Roles, and Department affiliations.
2. **Academic Structure:** Manages Semesters, Classes, and Enrollments.
3. **Project Context:** The central cluster linking Topics, Projects, and Team formations.
4. **Collaboration Features:** Stores high-volume data such as Tasks, Chat history, Whiteboard states, and Meeting logs.
5. **Assessment & Grading:** Records submissions, lecturer grades, and peer review metrics.

3.2. Authentication & User Management

FR-01: Login/Logout

- **Description:** Users log in to the system using School Email (Google OAuth) or provided credentials.

- **Actor:** All users.

Activity Diagram for Login Flow

Register Screen

Login Screen

FR-02: User Profile Management

- **Description:** Users can update personal information, avatars, and skills (for students to support team matching).
- **Actor:** All users.

3.3. Project & Team Administration

FR-03: Topic Assignment

- **Description:** Lecturers create project topics and assign them to student teams. Student leaders can also register for a topic provided by the lecturer.
- **Actor:** Lecturer, Student Leader.

FR-04: Team Organization

- **Description:** Student Leaders/Lecture can create a team and invite other members via email/system username. Members accept the invitation to join the workspace.
- **Actor:** Student, Lecture.

Team management view

Kick Team Member

Team Member'sView

FR-05: Task Management (Kanban Board)

- **Description:** Allows creating and assigning tasks to members, dragging and dropping task status (To Do, Doing, Done), and setting deadlines.

- **Actor:** Student (Team Member/Leader).

3.4. Real-time Collaboration Workspace

FR-06: Team Chat

- **Description:** Integrated chat system within each project. Supports sending text, images, and attachments. Messages are synchronized instantly (Real-time).

- **Actor:** Student, Lecturer.

Sequence Diagram for Real-time Messaging

Chat Interface

FR-07: Interactive Whiteboard

- **Description:** Members can draw diagrams and write notes on a shared whiteboard. All strokes are instantly synchronized to other online members (using [Socket.IO](#)).

- **Actor:** Student.

FR-08: Video Conference

- **Description:** Integrated video/audio calling and screen sharing for periodic team meetings or reviews with lecturers (using WebRTC).

- **Actor:** Student, Lecturer.

3.5. AI Assistant Integration

FR-09: AI Chatbot Support

- **Description:** A chatbot that assists in answering questions about project procedures, suggesting learning materials, or proposing topic ideas.
- **Actor:** Student.

3.6. Evaluation & Assessment

FR-10: Submission

- **Description:** The Team Leader submits reports, source code, or product links according to configured milestones.
- **Actor:** Student Leader.

Submit Using Zip Files

Submit Using GitHub Repository Link

FR-11: Peer Review

- **Description:** Team members evaluate each other's contributions at the end of the semester or each phase to ensure fairness.
- **Actor:** Student.

Activity Diagram for Peer Review Process

FR-12: Grading & Feedback

- **Description:** Lecturers view deliverables, grade according to criteria (Rubric), and provide detailed comments for each team.
- **Actor:** Lecturer.

3.7. Administrative & Academic Management

FR-13: Semester & Class Management (Staff)

- **Description:** Academic Staff can create new semesters, configure start/end dates, and import student lists (Excel/CSV) into classes.
- **Actor:** Staff.

FR-14: Department Statistics & Monitoring (Head of Department)

- **Description:** Head of Department can view a dashboard summarizing pass/fail rates, project quality, and lecturer performance statistics.
- **Actor:** Head of Department.

FR-15: System Configuration (Admin)

- **Description:** Administrators can manage technical settings such as AI API Keys (AWS Bedrock), Email Server (SMTP), and User Roles permissions.
- **Actor:** Admin.

General Settings

Security Settings

4. Non-Functional Requirements

4.1. Performance

- **Latency:** Latency for Real-time features (Chat, Whiteboard) must be under 200ms under stable network conditions.
- **Capacity:** The system supports a minimum of 50 concurrent users during the Demo/Defense phase.

4.2. Security

- **Authentication:** User passwords must be hashed in the database. Use JWT (JSON Web Token) for API authentication.
- **Data Privacy:** Grades and personal evaluation data (Peer review) must be confidential, visible only to the Lecturer and the evaluatee (or anonymous depending on configuration).

4.3. Reliability & Availability

- The system operates 24/7.
- Data (Submission files, critical messages) must be backed up periodically.

4.4. User Interface

- The interface is designed in a modern, Clean & Minimalist style.
- **Responsive Design:** Displays well on Laptops and Tablets (to serve Whiteboard drawing).

5. Requirement Appendix

5.1. Business Rules

BR-01: Peer Review Scoring Formula: The individual score of a student is calculated based on the Group Score given by the Lecturer and the Contribution Percentage evaluated by team members.

- **Formula:** Individual Score = Group Score * (Average Peer Rating / 10)
- **Condition:** The Peer Rating is on a scale of 1 to 10. If the calculated score exceeds 10, it is capped at 10.
- *Example:* Group gets 9. Student A gets an average Peer Rating of 8/10 from teammates. -> Score = $9 * 0.8 = 7.2$.

BR-02: Project Archival Policy

- When a semester ends (based on the academic timeline), all projects in that semester are automatically moved to "Archived" status.
- In "Archived" status, Workspace features (Chat, Whiteboard, Video Call) are disabled (Read-only mode) to save server resources.

5.2. Permission Matrix

This table defines the access levels for all user roles defined in the Project Scope (Section 3.2.b of the Proposal).

Roles Definition:

- **Student:** Can be a Team Leader or Member.
- **Lecturer:** Instructors who manage classes and projects.
- **Head of Dept:** Oversees the quality of topics and general statistics.
- **Staff:** Supports administrative tasks (managing student lists, semesters).
- **Admin:** System administrator (technical configurations).

Feature Category	Action / Permission	Student (Member)	Student (Leader)	Lecturer	HoD	Staff	Admin
System Access	Login / Logout	✓	✓	✓	✓	✓	✓
	View Dashboard (Statistics)	✗	✗	✓ (Class)	✓ (Dept)	✗	✓ (System)
User Management	Manage Users (Add/Edit/Ban)	✗	✗	✗	✗	✓	✓
	Import Student List (Excel)	✗	✗	✗	✗	✓	✗

Academic Management	Manage Semesters / Deadlines	✗	✗	✗	✓	✓	✗
	Approve/Manage Topic Bank	✗	✗	✓	✓	✗	✗
Project & Team	Create Team	✗	✓	✗	✗	✗	✗
	Invite Members	✗	✓	✗	✗	✗	✗
	Assign Topic to Team	✗	✓ (Register)	✓ (Approve)	✗	✗	✗
Collaboration	Use Whiteboard (Draw/Edit)	✓	✓	✓ (View)	✗	✗	✗
	Chat / Video Call	✓	✓	✓ (Join)	✗	✗	✗
	Use AI Assistant (Chatbot)	✓	✓	✗	✗	✗	✗
	Create/Edit Tasks (Kanban)	✓	✓	✓ (View)	✗	✗	✗
Resources	Upload Learning Materials	✗	✗	✓	✓	✗	✗
	Download/View Materials	✓	✓	✓	✓	✗	✗
Assessment	Submit Project Deliverables	✗	✓	✗	✗	✗	✗
	Perform Peer Review	✓	✓	✗	✗	✗	✗
	Grade Projects & Feedback	✗	✗	✓	✗	✗	✗
	View Final Grades	✓	✓	✓	✓	✓	✗

(Symbol Key:  = Allowed,  = Denied)

IV. Software Design Description

1. System Design

1.1 System Architecture

The CollabSphere system is designed based on the Client-Server model with a Modular Monolith (or Microservices-ready) architecture. It ensures a clear separation of concerns, high performance, and scalability, specifically supporting real-time communication and AI integration.

Key Components:

- **Frontend (Client):**

- o Built with **ReactJS** (using **Vite** as the build tool).
- o UI Library: **Ant Design** or **MUI**.
- o Communicates with the Backend via **RESTful API** (Axios) for standard requests and **WebSocket** ([Socket.IO](#) Client) for real-time features.

- **Backend (Server):**

- o Built with **Python FastAPI**, running on **Uvicorn**.
- o Handles core business logic: Authentication, Project Management, Team Management, and AI Integration.
- o **Socket.IO Server**: Integrated directly into FastAPI (using python-socketio AsyncServer) to manage Chat rooms and Notifications.

- **Database & Storage:**

- o **PostgreSQL**: The primary relational database for storing Users, Projects, Teams, etc.
- o **Redis**: Acts as a **Message Broker** for [Socket.IO](#) (Pub/Sub) and a caching layer for session data.

o **Cloudinary:** Stores media files (images, documents); the database only stores the secure URLs.

· **External Services:**

o **Google Gemini API:** Provides AI-powered features (Idea suggestion, description refinement).

o **Google OAuth:** Handles user authentication via CollabSphere email.

1.2 Package Diagram

1.2.1 Frontend Structure (Client-side)

No	Package/Folder	Description
1	src/api	Contains Axios client configurations and API service functions (Service Proxy) to communicate with the Backend.
2	src/assets	Stores static resources such as images, fonts, icons, and global stylesheets.
3	src/components	Contains reusable UI components (Shared Components) like Buttons, Modals, Inputs, and Layouts.
4	src/context	Manages Global State using React Context API (e.g., AuthContext, SocketContext).
5	src/hooks	Contains Custom Hooks to separate logic from UI (e.g., useAuth, useSocket, useDebounce).
6	src/pages	Contains the main application screens (e.g., Dashboard, ProjectList, TeamWorkspace, ChatRoom).
7	src/services	Contains client-side business logic and complex data processing before rendering or sending requests.

8	src/socket	Configures the Socket.IO Client, defining Event Listeners and Emitters for Chat and Notifications.
9	src/utils	Helper functions (e.g., Date formatting, Data validation, String processing).
10	src/App.jsx	The Root Component, configuring Routing (React Router) and Context Providers.

1.2.2 Backend Structure (Server-side)

No	Package / File	Description
1	app/main.py	Application Entry Point. This file initializes the FastAPI instance, configures CORS (Cross-Origin Resource Sharing) middleware, and mounts the Socket.IO (ASGI) application to handle real-time connections.
2	app/api/	Controllers Layer. Contains sub-packages for API Routes (e.g., v1/auth, v1/projects). It handles HTTP requests, dependency injection, and returns responses. No complex logic should be placed here.
3	app/core/	Configuration & Security. Contains config.py (loads environment variables like DB URL, Gemini Key) and security.py (handles JWT generation, Password hashing with Bcrypt).
4	app/models/	Data Access Layer (ORM). Defines SQLAlchemy classes that map directly to PostgreSQL tables (e.g., User, Team, Project).
5	app/schemas/	Data Transfer Objects (DTOs). Defines Pydantic models used for data validation. It separates the internal database structure (models) from the data sent/received via API (schemas).
6	app/services/	Business Logic Layer. Contains the core logic of the application. - ai_service.py: Handles Google Gemini API calls (Prompts & Responses). - socket_service.py: Logic for broadcasting real-time messages. - crud_service.py: Reusable DB operations.

7	app/socket/	Real-time Event Handlers. Specifically manages Socket.IO events such as @sio.on('join_room'), @sio.on('send_message'). It acts similarly to a "Controller" but for WebSocket events.
8	app/db/	Database Connectivity. • session.py: Configures the PostgreSQL connection engine and session local. • redis.py: Configures the Redis connection for caching and Pub/Sub.
9	app/utils/	Utilities. Helper functions reused across the system (e.g., File upload helpers for Cloudinary, random ID generators, Date formatting tools).
10	requirements.txt	Dependencies. Lists all Python libraries required to run the project (fastapi, uvicorn, sqlalchemy, python-socketio, google-generativeai).

2. Database Design

2.1 User & Authentication

- **PK:** Primary Key
- **FK:** Foreign Key
- **U:** Unique
- **NN:** Not Null
- **AI:** Auto Increment

2.1.1 User Table

No.	Field Name	Data Type	Constraint	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for user
2	Username	varchar(255)	NN, U	Login username
3	Password	varchar(255)	NN	Encrypted password

4	Code	varchar(255)	U	User code (Student ID/Staff ID)
5	Cache	varchar(255)		Latest role of user for session
6	Gender	varchar(255)		Gender (Male/Female/Other)
7	Avatar	varchar(255)		Profile picture URL
8	FirstName	varchar(255)	NN	User's first name
9	LastName	varchar(255)	NN	User's last name
10	Email	varchar(255)	NN, U	Email address
11	Dob	date		Date of birth
12	Phone	varchar(255)		Phone number
13	Address	varchar(255)		Physical address
14	Department	varchar(255)		Department name
15	CreatedDate	datetime	NN	Creation date
16	CreatedBy	varchar(255)		Who created this record
17	UpdatedDate	datetime		Last modification date
18	UpdatedBy	varchar(255)		Who last updated this record
19	IsDeleted	bit	NN	Delete flag (0=active, 1=deleted)
20	Note	varchar(255)		Additional notes

2.1.2 Role Table

No.	Field Name	Data Type	Constraint	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for role
2	RoleName	BIGINT	FK, NN	Identifier linking to the account table

3	CreatedDate	date	NN	Creation date
4	CreatedBy	varchar(255)		Who created this record
5	UpdatedDate	date		Last modification date
6	UpdatedBy	varchar(255)		Who last updated
7	IsDeleted	bit	NN	Delete flag (0=active, 1=deleted)
8	Note	varchar(255)		Additional notes

SUMMARY OF THE 4 ROLES

Role Name	Core Responsibilities in CollabSphere System
Admin	System maintenance, User account management (View/Deactivate), Monitoring system reports
Head Of Department	Approving/Denying project ideas, assigning projects to classes, and managing academic criteria
Staff	Managing academic data: Importing Semester, Timeline, Subject, Syllabus, Class, and Student/Lecturer lists.
Lecturer/ Mentor	Proposing ideas, mentoring teams, grading milestones/checkpoints, and providing evaluations
Student	Forming teams, executing projects, managing tasks in workspace, and performing peer reviews

2.1.3 UserXRole

No.	Field Name	Data Type	Constraint	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for mapping record
2	UserId	varchar(255)	FK, NN	Reference to User.Id

3	RoleId	varchar(255)	FK, NN	Reference to Role.Id
4	SemesterId	varchar(255)	FK, NN	Reference to Semester
5	IsPrimary	bit	NN	Primary role of user in current semester
6	CreatedDate	date	NN	Creation date
7	CreatedBy	varchar(255)		Who created
8	UpdatedDate	date		Last modification date
9	UpdatedBy	varchar(255)		Who last updated
10	IsDeleted	bit	NN	Delete flag (0=active, 1=deleted)
11	Note	varchar(255)		Additional notes

2.1.4 Refresh Token Table

No	Field Name	Data Type	Constraint	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for the token record (UUID)
2	Token	text	NN	The actual refresh token string generated by the system.
3	Expires	datetime	FK, NN	The expiration date and time of the token.
4	UserId	varchar(255)		Links to User.Id who owns this token.
5	CreatedByIp	varchar(50)		The IP address from which the token was issued.
6	Revoked	datetime		The time when the token was manually revoked (if any).
7	RevokedByIp	varchar(50)		The IP address from which the token was revoked.
8	ReplacedBy	varchar(255)		Stores the new token ID if this token was rotated.
9	IsDeleted	bit	NN	Soft delete flag (0: Active, 1: Deleted).

2.2 Semester & Timeline

No.	Field Name	Data Type	Constraint	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for the semester record.
2	CriteriaFormId	varchar(255)	FK	Reference to the evaluation criteria form.
3	SemesterCode	varchar(255)	NN, U	Internal semester code (e.g., SP25).
4	SemesterName	varchar(255)	NN	Full name (e.g., Spring 2025).
5	SemesterPrefixName	varchar(255)		Prefix string for generating topic codes.
6	StartDate	date	NN	Official start date of the semester.
7	EndDate	date	NN	Official end date of the semester
8	PublicTopicDate	date		Date when the topic list is published to students
9	LimitTopicOnlyMentor	integer(10)		Max topics a lecturer can mentor alone
10	LimitTopicSubMentor	integer(10)		Max topics a lecturer can co-mentor
11	CreatedDate	date	NN	Date the record was created.
12	CreatedBy	varchar(255)		Staff ID who created this semester.
13	UpdatedDate	date		Date of the last modification.
14	UpdatedBy	varchar(255)		User ID who last updated the record.

2.3 Profession Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for profession
2	ProfessionName	varchar(225)	NN	Name of the profession (e.g., IT, Business)
3	CreatedDate	date	NN	Record creation date

4	CreatedBy	varchar(255)		User ID who created this record
5	UpdatedDate	date		Last modification date
6	UpdatedBy	varchar(255)		User ID who last updated
7	IsDeleted	bit	NN	0: Active, 1: Deleted
8	Note	varchar(255)		Additional academic notes ¹

2.4 Specialty Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for specialty ¹
2	ProfessionId	varchar(255)	FK, NN	Reference to parent Profession.Id
3	SpecialtyName	varchar(255)	NN	Name of the specialty
4	CreatedDate	date	NN	Record creation date
5	CreatedBy	varchar(255)		User ID who created
6	UpdatedDate	date		Last modification date
7	UpdatedBy	varchar(255)		User ID who last updated
8	IsDeleted	bit	NN	0: Active, 1: Deleted
9	Note	varchar(255)		Additional notes

2.5 Subject Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for subject
2	SubjectCode	varchar(255)	NN, U	Academic code
3	SubjectName	varchar(255)	NN	Full name of the subject

4	ProfessionId	varchar(255)	FK, NN	Reference to Profession.Id
5	CreatedDate	date	NN	Record creation date
6	UpdatedDate	date		Last modification date
7	IsDeleted	bit	NN	0: Active, 1: Deleted

2.6 Class Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for class
2	SemesterId	varchar(255)	FK, NN	Reference to Semester
3	SubjectId	varchar(255)	FK, NN	Reference to Subject.Id
4	ClassCode	varchar(255)	NN, U	Unique class code
5	ClassName	varchar(255)	NN	Display name of the class
6	LecturerId	varchar(255)	FK	Main lecturer assigned to class
7	CreatedDate	date	NN	Record creation date
8	IsDeleted	bit	NN	0: Active, 1: Deleted

2.7 Criteria Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for the criteria.
2	CriteriaFormId	varchar(255)	FK, NN	Reference to the parent CriteriaForm.
3	Question	varchar(255)	NN	The specific question or rubric name for evaluation.
4	ValueType	varchar(255)	NN	Data type of the answer (e.g., Integer, String, Boolean).

5	CreatedDate	date	NN	Record creation date.
6	CreatedBy	varchar(255)		User ID of the creator (Admin/Head Dept/Staff).
7	UpdatedDate	date		Last modification date.
8	UpdatedBy	varchar(255)		User ID who last updated the record.
9	IsDeleted	bit	NN	Delete flag (0: active, 1: deleted).
10	Note	varchar(255)		Additional instructions or notes for this criteria.

2.7.1 Criteria Form Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for the criteria form (UUID).
2	Title	varchar(255)	NN	Title of the criteria form (e.g., Final Presentation Form).
3	CreatedDate	date	NN	Record creation date.
4	CreatedBy	varchar(255)		User ID who created this form (Staff or Head Dept).
5	UpdatedDate	date		Last modification date
6	UpdatedBy	varchar(255)		User ID who last updated the form
7	IsDeleted	bit	NN	Soft delete flag (0: active, 1: deleted)
8	Note	varchar(255)		Additional notes or instructions for the form

2.7.2 CriteriaXCriteria Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for this mapping record (UUID)

2	Criteriald	varchar(255)	FK, NN	Reference to Premium Domains Own Your Digital Legacy
3	CriteriaFormId	varchar(255)	FK, NN	Reference to CriteriaForm.Id
4	CreatedDate	date	NN	Date when the criteria was added to the form
5	CreatedBy	varchar(255)		User ID who performed the mapping
6	UpdatedDate	date		Timestamp of the last update to this mapping
7	UpdatedBy	varchar(255)		User ID who updated the record
8	IsDeleted	bit	NN	Delete flag (0: active, 1: removed from form)
9	Note	varchar(255)		Additional notes about this specific placement

2.7.3 Answer Criteria Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for the answer record (UUID).
2	IdeaVersionRequestId	varchar(255)	FK, NN	Reference to the specific IdeaVersionRequest.Id .
3	Criteriald	varchar(255)	FK, NN	Reference to the specific Premium Domains Own Your Digital Legacy being graded.
4	Value	varchar(255)		The actual score or comment provided by the reviewer.
5	CreatedDate	date	NN	Record creation date.
6	CreatedBy	varchar(255)		User ID of the reviewer who provided the answer.
7	UpdatedDate	date		Last modification date.

8	UpdatedBy	varchar(255)		User ID who last updated the answer.
9	IsDeleted	bit	NN	Soft delete flag (0: active, 1: deleted).
10	Note	varchar(255)		Additional private notes from the reviewer about this specific point.

2.8 Idea Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for idea (UUID).
2	OwnerId	varchar(255)	FK, NN	User ID who proposed the idea (Student or Lecturer)
3	MentorId	varchar(255)	FK	Lecturer ID who acts as the primary mentor
4	SubMentorId	varchar(255)	FK	Lecturer ID who acts as the co-mentor.
5	SpecialtyId	varchar(255)	FK, NN	Reference to http://Specialty.Id for department classification.
6	Type	varchar(255)		Type/Category of the idea.
7	Status	varchar(255)	NN	Pending, Approved, Rejected, ConsiderByMentor, ConsiderByCouncil
8	IsExistedTeam	bit	NN	Indicates if the idea already has a pre-formed team (0: No, 1: Yes).
9	IsEnterpriseTopic	bit	NN	Indicates if the idea is sponsored by an enterprise (0: No, 1: Yes).
10	CreatedDate	date	NN	Submission date of the idea.
11	CreatedBy	varchar(255)		User ID who created the record.
12	UpdatedDate	date		Last modification date.
13	UpdatedBy	varchar(255)		User ID who last updated the record.

14	IsDeleted	bit	NN	Soft delete flag (0: active, 1: deleted).
15	Note	varchar(255)		Additional notes/comments from mentors or council

2.8.1 Idea Version Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for idea version (UUID).
2	Ideald	varchar(255)	FK, NN	Reference to the parent Idea.Id .
3	StageIdeald	varchar(255)	FK	Reference to the specific stage of the idea.
4	Version	integer(10)	NN	Version number (e.g., 1, 2, 3).
5	Description	varchar(255)		Detailed summary of this version.
6	Abbreviations	varchar(255)		Short name or acronym for the idea.
7	VietNamName	varchar(255)		Official name of the idea in Vietnamese.
8	EnglishName	varchar(255)		Official name of the idea in English.
9	File	varchar(255)		URL to the uploaded document file (Cloudinary)
10	TeamSize	integer(10)		Required number of members for this version.
11	CreatedDate	date	NN	Date this version was created.
12	CreatedBy	varchar(255)		User ID who created this version.
13	UpdatedDate	date		Last modification date of this version.
14	UpdatedBy	varchar(255)		User ID who performed the update.
15	IsDeleted	bit	NN	Soft delete flag (0: active, 1: deleted).
16	Note	varchar(255)		Additional remarks for this version.

2.8.2 Idea Version Request Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for idea version request (UUID)
2	IdeaVersionId	varchar(255)	FK	Reference to the specific IdeaVersion.Id being reviewed
3	ReviewerId	varchar(255)	FK	User ID of the person reviewing the request (Lecturer/Council)
4	CriteriaFormId	varchar(255)	FK	Reference to CriteriaForm.Id used for this review
5	ProcessDate	date		The date when the reviewer processed the request
6	Status	varchar(255)		Current status (e.g., Pending, Approved, Rejected)
7	Role	varchar(255)		Role of the respondent for this request (Mentor or Council)
8	CreatedDate	date	NN	Record creation date
9	CreatedBy	varchar(255)		User ID who created the request
10	UpdatedDate	date		Last modification date
11	UpdatedBy	varchar(255)		User ID who last updated the record
12	IsDeleted	bit	NN	Soft delete flag (0: active, 1: deleted)
13	Note	varchar(255)		Additional notes or feedback from the reviewer

2.9 Topic Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for topic (UUID).
2	IdeaVersionId	varchar(255)	FK, NN	Reference to the approved IdeaVersion.Id .

3	TopicCode	varchar(255)	NN, U	Official academic code for the topic (e.g., SE1801_Topic01).
4	CreatedDate	date	NN	Date the topic was officially created.
5	CreatedBy	varchar(255)		User ID who created the record.
6	UpdatedDate	date		Last modification date.
7	UpdatedBy	varchar(255)		User ID who last updated the record.
8	IsDeleted	bit	NN	Soft delete flag (0: active, 1: deleted).
9	Note	varchar(255)		Additional academic notes.

2.9.1 Topic Version Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for topic version (UUID).
2	TopicId	varchar(255)	FK, NN, U	Reference to the parent Topic.Id .
3	FileUpdate	varchar(255)		URL to updated documentation after review (Cloudinary).
4	ReviewStage	integer(10)		Indicates the review round (e.g., 1st or 2nd review).
5	Status	varchar(255)	NN	Status of the version (Pending, Approved, Rejected).
6	Comment	varchar(255)		Evaluation comments regarding this version.
7	CreatedDate	date	NN	Date this version was submitted.
8	CreatedBy	varchar(255)		User ID who submitted the update.
9	IsDeleted	bit	NN	Soft delete flag (0: active, 1: deleted).

2.9.2 Topic Version Request Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for request record (UUID).
2	TopicVersionId	varchar(255)	FK, NN	Reference to the TopicVersion.Id being requested
3	ReviewerId	varchar(255)	FK	User ID of the reviewer (Mentor/Manager).
4	ProcessDate	date		Date when the request was processed.
5	Status	varchar(255)	NN	Final decision status.
6	Role	varchar(255)		Role of the respondent (Mentor or Manager).
7	Feedback	varchar(255)		Feedback from the reviewer to the team.
8	CreatedDate	date	NN	Request creation date.
9	IsDeleted	bit	NN	Soft delete flag (0: active, 1: deleted).

2.10 Project Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for project (UUID)
2	LeaderId	varchar	FK	Reference to User who is leader of team
3	TopicId	varchar(255)	FK	Reference to official Topic
4	TeamName	varchar(255)		Name of the team
5	TeamCode	varchar(255)		Unique code of the team
6	Status	varchar(255)		Status of the project team
7	TeamSize	integer(2)		Number of members in team
8	DefenseStage	integer(10)		Current defense stage
9	CreatedDate	date	NN	Date project was initialized

10	CreatedBy	varchar(255)		User ID who created the record
11	UpdatedDate	date		Last modification date
12	UpdatedBy	varchar(255)		User ID who last updated
13	IsDeleted	bit	NN	Delete flag (0=active, 1=deleted)
14	Note	varchar(255)		Additional notes

2.11 Team Member Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique ID for team member record
2	UserId	varchar(255)	FK	Reference to User (Student)
3	ProjectId	varchar(255)	FK	Reference to Project
4	Role	varchar(255)		Role within the team (e.g., Developer, QA)
5	JoinDate	date		Date joining the team
6	LeaveDate	date		Date leaving the team (if applicable)
7	Status	varchar(255)		Member status (Active/Inactive)
8	Attitude	varchar(255)		Evaluation of member's attitude
9	MentorConclusion	varchar(255)		Final feedback from Mentor for the member
10	CommentDefense1	varchar(255)		Mentor's comment in defense stage 1
11	CommentDefense2	varchar(255)		Mentor's comment in defense stage 2
12	CreatedDate	date	NN	Record creation date
13	IsDeleted	bit	NN	Delete flag

2.12 Review Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique ID for review session
2	ProjectId	varchar(255)	FK	Reference to Project
3	Reviewer1Id	varchar(255)	FK	Reference to 1st Reviewer (Lecturer)
4	Reviewer2Id	varchar(255)	FK	Reference to 2nd Reviewer (Lecturer)
5	Number	integer(10)		Review session number (1, 2, or 3)
6	ReviewDate	date		Scheduled date for review
7	Room	varchar(255)		Location/Room for the review
8	Slot	varchar(255)		Time slot for the review
9	Description	varchar(255)		Summary or goals of the review session
10	FileUpload	varchar(255)		URL to documents uploaded for review
11	IsDeleted	bit	NN	Delete flag

2.13 Invitation Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Invitation unique identifier
2	ProjectId	varchar(255)	FK	Reference to the Project
3	SenderId	varchar(255)	FK	User who sends the invitation
4	ReceiverId	varchar(255)	FK	User who receives the invitatio
5	Status	varchar(255)		Pending, Accepted, Rejected
6	Type	varchar(255)		Type of invitation
7	Content	varchar(255)		Message content of the invitation

8	CreatedDate	date	NN	Date invitation was sent
---	-------------	------	----	--------------------------

2.14 Feedback Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for mentor feedback (UUID).
2	ProjectId	varchar(255)	FK, NN, U	Reference to Project.Id . Each project has one final mentor feedback.
3	ThesisContent	varchar(255)		Feedback on thesis content compared to research objectives.
4	ThesisForm	varchar(255)		Feedback on layout, presentation, English, and citations.
5	AchievementLevel	varchar(255)		Level of achievement compared to the original plan.
6	Limitation	varchar(255)		Identified limitations or weaknesses of the project.
7	CreatedDate	date	NN	Record creation date.
8	CreatedBy	varchar(255)		User ID of the Mentor who created the feedback.
9	UpdatedDate	date		Timestamp of the last modification.
10	UpdatedBy	varchar(255)		User ID of the person who last updated the record.
11	IsDeleted	bit	NN	Soft delete flag (0: Active, 1: Deleted).
12	Note	varchar(255)		Additional academic notes or private comments.

2.15 Notification Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for notification (UUID).
2	UserId	varchar(255)	FK, NN	Reference to the User.Id who receives the notification.
3	ProjectId	varchar(255)	FK	Reference to the Project.Id context (if applicable).
4	Description	varchar(255)	NN	The content/message of the notification.
5	Type	varchar(255)		Category: System, Team, Evaluation, Resource, etc.
6	Role	varchar(255)		Purpose of notification targeted at a specific role.
7	IsRead	bit	NN	Read status (0: Unread, 1: Read).
8	CreatedDate	date	NN	Date and time when the notification was generated
9	CreatedBy	varchar(255)		User ID or System Process that triggered the notification.
10	UpdatedDate	date		Timestamp of the last update (e.g., when marked as read).
11	UpdatedBy	varchar(255)		User ID who performed the update.
12	IsDeleted	bit	NN	Soft delete flag (0: Active, 1: Deleted).
13	Note	varchar(255)		Additional metadata or developer notes.

2.16 Message Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for each message (UUID)
2	SenderId	varchar(255)	FK, NN	Reference to User.Id of the person who sent the message

3	ProjectId	varchar(255)	FK, NN	Reference to Project.Id to identify the team chat context
4	Content	text	NN	The text content of the message
5	MessageType	varchar(50)		Type of message: 'Text', 'File', 'Image', 'System'
6	FilePath	varchar(255)		URL to the uploaded file/image if the message contains resources
7	IsRead	bit	NN	Status of the message: 0: Unread, 1: Read.
8	CreatedDate	datetime	NN	Timestamp when the message was sent
9	CreatedBy	varchar(255)		Redundant reference to SenderId for audit consistency.
10	UpdatedDate	datetime		Timestamp of the last update (e.g., edited message).
11	UpdatedBy	varchar(255)		User ID who performed the update.
12	IsDeleted	bit	NN	Soft delete flag (0: Active, 1: Deleted).
13	Note	varchar(255)		Additional metadata for specialized message types.

2.17 Grading / Rate Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for the grade record (UUID).
2	ProjectId	varchar(255)	FK, NN	Reference to Project.Id .
3	StudentId	varchar(255)	FK, NN	Reference to User.Id of the student being graded.
4	ReviewerId	varchar(255)	FK, NN	Reference to User.Id of the person giving the grade.
5	Score	decimal(5,2)	NN	The numerical grade/rating value (e.g., 0.00 to 10.00).

6	Comment	text		Overall qualitative feedback/feedback for the student.
7	GradeType	varchar(50)	NN	Category: 'PeerReview', 'MentorGrade', 'CouncilGrade', 'Final'.
8	Status	varchar(50)		Status: 'Draft', 'Finalized', 'Published'.
9	IsFinal	bit	NN	1 if this is the official final grade, 0 otherwise.
10	CreatedDate	datetime	NN	Timestamp when the grade was first recorded.
11	CreatedBy	varchar(255)		User ID who created the record.
12	UpdatedDate	datetime		Timestamp of the last modification.
13	UpdatedBy	varchar(255)		User ID who last updated the grade.
14	IsDeleted	bit	NN	Soft delete flag (0: Active, 1: Deleted).
15	Note	varchar(255)		Additional administrative notes.

2.18 Capstone Schedule Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for capstone schedule
2	ProjectId	varchar(255)	FK, NN	Reference to associated Project.Id
3	Time	varchar(255)	NN	Specific presentation time (e.g., 08:30 AM)
4	Date	date	NN	Presentation date
5	HallName	varchar(255)		Physical or Virtual location (e.g., Hall A, Zoom link)
6	Stage	integer(10)	NN	Defense stage (e.g., 1: Midterm, 2: Final)
7	CreatedDate	date	NN	Record creation date
8	CreatedBy	varchar(255)		User ID who created the schedule (Staff/Admin)

9	UpdatedDate	date		Last modification date
10	UpdatedBy	varchar(255)		User ID who last updated the record
11	IsDeleted	bit	NN	Soft delete flag (0: Active, 1: Deleted)
12	Note	varchar(255)		Additional notes for the defense session

2.19 Workspace Task & AI Assistant

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier for the task.
2	ProjectId	varchar(255)	FK, NN	Reference to Project.Id .
3	TaskName	varchar(255)	NN	Title of the task/exercise.
4	Description	text		Detailed instructions.
5	IsAiGenerated	bit	NN	1: Generated by AI Assistant, 0: Manual.
6	AiPromptSource	text		The prompt used to generate this task
7	Status	varchar(50)		Task status (To-do, Doing, Done)
8	Priority	varchar(50)		Low, Medium, High.
9	Assigneeld	varchar(255)	FK	Student assigned to this task

2.10 Syllabus Table

No.	Field Name	Data Type	Constraints	Description
1	Id	varchar(255)	PK, NN, U	Unique identifier (UUID).
2	SubjectId	varchar(255)	FK, NN	Reference to Subjects.Id .
3	SyllabusName	varchar(255)	NN	Name (e.g., "Syllabus PRN231 v2.0").

4	Description	text		Summary of course objectives.
5	DecisionNo	varchar(100)		Official approval decision number.
6	EffectiveDate	date	NN	Date the syllabus becomes valid.
7	ScoringScale	int		Grading scale (e.g., 10 or 100).
8	MinScoreToPass	int		Minimum score required to pass.
9	IsActive	bit	NN	Status (1: Active, 0: Inactive).
10	CreatedDate	datetime	NN	Record creation timestamp.
11	CreatedBy	varchar(255)		Staff ID who created it.
12	UpdatedDate	datetime		Last modification timestamp.
13	UpdatedBy	varchar(255)		Last modifier ID.
14	IsDeleted	bit	NN	Soft delete (0: Active, 1: Deleted).
15	Note	varchar(255)		Additional administrative remarks.

3. Design Detail

3.1. Authentication

3.1.1 login

3.1.1.1 Class Diagram

3.1.1.2 Sequence Diagram

3.2. User Manager

3.2.1. Class Diagram

3.2.2 Sequence Diagram

3.2.2.1 Admin CRUD Users, Ban/Unban

3.2.2.2 Staff Import Students (gv tạo ra ds lớp và từ đó lấy ds lớp)

3.2.2.3. Leturer Assign Role

3.3. Topic Management

3.3.1. Class Diagram

3.3.2. Sequence Diagram

3.3.2.1. Student Create Idea

3.3.2.2 HoD Evaluate Idea (duyệt idea)

3.3.2.3 Council Evaluate Topic

3.3.2.4 Student Resubmit Idea

3.3.2.5. Lecturer Create Idea

3.3.2.6. Lecturer Resubmit Idea

3.3.2.7. Student Update Topic

3.3.2.8. Lecturer/Mentor Respond Topic Update

3.3.2.9. Lecturer Respond Topic Update

3.4. Team Management

3.4.1. Class Diagram

3.4.2. Sequence Diagram

3.4.2.1 Student Create Team

3.4.2.2. Leader Invite + Student Respond

3.4.2.3. Student Request to Join + Leader Respond

3.4.2.4. Leader Rate Member

3.4.2.5. Lecturer/Mentor Give Conclusion

3.5. Review & Defense

3.5.1 Class Diagram

3.5.2. Sequence Diagram

3.5.2.1. Lecturer Import Review

3.5.2.2. Student Submit Checklist

3.5.2.3. Lecturer Import Defense

3.6. Semester, Timeline, Criteria

3.6.1. Class Diagram

3.6.2. Sequence Diagram

3.6.2.1. Semester, Timeline, CriteriaForm

3.6.2.2. Staff Manage Semester + Timeline

3.6.2.3. Staff Manage Criteria Form

3.7. Notification & Messaging

3.7.1. Class Diagram

3.7.2. Sequence Diagram

3.7.2.1. Real-time Chat ([Socket.IO](#))

3.7.2.2. System Notification

3.8. AI Assistant (Google Gemini API)

3.8.1. Class Diagram

3.8.2. Sequence Diagram

3.8.2.1. AI helping Student

3.8.2.2 Lecturer generate assignment with AI

3.8.2.3. AI review documents

V. Software Testing Documentation

1. Scope Of Testing

1.1 Student Role Test Features

Account Management: Test successful login/registration, authentication error handling, and password recovery functionality

Personal Profile: Test the ability to view, edit personal information.

Idea Management: Check the process of creating new ideas, viewing the list, idea details, and verifying the display of the Approval Page status

Team Management: Test the ability to create teams, manage members (add/edit), update team info, and submit team/project forms (Form Submit)

Team Evaluation: Verify that the function for evaluating (Rating) team members operates accurately and stores the assessment scores.

Review & Feedback: Check the ability to view, upload, and edit reviews, as well as view detailed feedback (Feedback Review Detail) on submitted reviews.

Notifications: Test the receiving, displaying of the list, and viewing the details of system notifications related to teams/ideas.

Real-time Workspace:

- Verify the integrated view functionality: Split screen showing Chat (Right), Whiteboard/Task (Center), and Video Call Header (Top).
- Test real-time data synchronization between components when other members make changes

Kanban Board Detail:

- Test drag-and-drop functionality for Task Cards across columns (To Do, Doing, Done).

- Verify task creation through modals and the correct display of tags such as Priority and Due Date.

Whiteboard Canvas:

- Test the responsiveness and accuracy of drawing tools: Pen, Shape, Note, and Eraser.
- Verify the Undo/Redo functions and the real-time visibility of other members' cursor presence.

Video Meeting Room:

- Check the webcam grid view display and toolbar functionality (Mute, Stop Video, Share Screen, Chat, Leave).
- Verify stable audio/video transmission and in-meeting chat features.

Submission Portal:

- Test the form for uploading zip files/documents or pasting GitHub links for project milestones.
- Verify that project status updates correctly upon successful submission.

Peer Review Form:

- Verify the rating matrix for evaluating teammates on Contribution, Attitude, and Skill.
- Test the functionality of sliding scales and comment text areas for detailed feedback.

1.2 Lecturer Role Test Features

Access & General: Verify successful login, homepage navigation, and the ability to access/view posts on the forum.

Dashboard & Feedback: Verify the (Dashboard} overview, the Lecturer's ability to accept student invitations, submit quality feedback for reports, and manage their personal profile.

Idea Management: Test the ability to view idea lists (from both Lecturers and Students), view idea details, and execute the idea approval process (Approval Idea).

Team & Project Management: Check the ability to access the team management interface, view project details, and manage/use the scheduling calendar.

Class Progress Monitoring:

- Verify the table view showing all teams within a class with specific columns: "Last Activity", "Completed Tasks", and "Submission Status".
- Test the ability to track real-time progress of each team's Kanban board from the lecturer's perspective.

Grading & Feedback Interface:

- Test the split-screen interface: verify the left side correctly displays student submissions (PDF viewer or Code viewer) while the right side shows the Rubric scoring system.
- Verify that the Rubric scoring accurately calculates the final grade based on weighted criteria.
- Check the functionality of the Feedback box for submitting detailed comments and suggestions to the teams.

Review Management: Test the Lecturer's ability to manage pending reviews, view review details, and submit specific feedback (Feedback-review) for those assessments.

Notifications: Check the receiving, displaying of the list, and viewing the details of system notifications.

Profile Management: Verify the ability to accept student invitations and manage personal profile information.

1.3 Admin Role Test Features

Login & Access Rights: Test for successful login and verify exclusive access rights to the Admin page for users with the Admin role only.

Dashboard Management: Verify correct access and display of overall metrics on the Manage dashboard, ensuring proper navigation functionality.

Basic User Management: Test the user list management interface, including the ability to Create User, view Detail user, and Update user information.

User Data Operations: Verify the ability to Remove user from the system and confirm that the bulk Import user function operates correctly.

System Configuration: Verify the interface for configuring AI API Keys (e.g., Gemini), Email Server (SMTP), and Cloudinary integration.

Notification Sending: Check that the Send Notification function works properly, allowing the Admin to send announcements to specific user groups or the entire system.

1.4 Staff Role Test Features

Data Import:

- Test the "Import files" function to automatically create and manage subjects, classes, syllabuses, and student/lecturer accounts from Excel/CSV.
- Verify the ability to view lists and details of subjects, classes, syllabuses, and accounts.

Resource Management: Verify the ability to view lists and details of subjects, classes, syllabuses, and accounts.

Class Assignment: Test the functionality of assigning lecturers and students to specific classes.

AI Assistance: Verify the "AI chat for assistance" feature for staff support using integrated AI models.

1.5 Head Department Role Test Features

System Oversight: Verify the ability to view lists and details of classes, subjects, syllabuses, and all projects (pending, approved, denied).

Project Approval: Test the workflow for approving or denying pending projects and updating approved projects.

Project Assignment: Verify the process of assigning projects from the approved list to all classes.

Statistics Dashboard: Verify the display of charts showing pass/fail rates, project quality distribution, and system resource usage.

Advanced AI Support: Test the AI chat functionality specifically tailored for department head assistance.

2. Testing Strategy and Levels

2.1 Unit Testing

Responsible: Developers.

Objective: Verify the correctness of individual modules (e.g., Cloudinary file upload logic, Gemini API prompt handling, and [Socket.IO](#) event emitters)

Technique: White-box Testing with automated tools:

- Frontend: Vitest (Testing React components and logic).
- Backend: Pytest (Testing FastAPI services and logic).

Focus: Ensure each utility function and API endpoint handler operates correctly in isolation, particularly the AI prompt construction and real-time event triggers.

Completion Criteria: All unit tests achieve a pass rate of at least 95%.

2.2 Integration Testing

Responsible: Development and QA teams.

Objective: Validate interactions between modules (e.g., Bulk Data Import processing, Backend API with PostgreSQL, Redis, and real-time services like [Socket.IO](#) , PeerJS/LiveKit, and Cloudinary).

Technique: API Testing using Postman and FastAPI TestClient.

Focus: Verify the seamless flow of data across the service-layered architecture, ensuring that database transactions, cache updates, and third-party API calls (Gemini/Cloudinary) function as a unified system.

Completion Criteria: All integration tests pass with no major defects

2.3 System Testing

Responsible: QA team

Objective: Confirm the entire system functions as expected, integrating all components and roles (Admin, Lecturer, Student, Staff, and Head Department), including AI-powered features and video conferencing.

Technique: Black-box Testing focusing on the User Interface (UI Testing) using Selenium WebDriver (Python) to script end-to-end scenarios for the React UI.

Focus: Validate the end-to-end user journeys, such as a Student creating a project in the Real-time Workspace or a Lecturer grading via the split-screen interface.

Completion Criteria: All functional and non-functional requirements are met, including usability and security.

2.4 Performance Testing

Responsible: QA team

Objective: Assess the system's behavior under various workloads, especially in real-time scenarios ([Socket.IO](#) chat concurrency, Redis caching speed, and PeerJS/LiveKit video stability).

Technique: Load Testing using Locust (Python-based scripting for scalability).

Focus: Stress test the backend to ensure the AI API integration and real-time collaboration tools do not degrade under pressure.

Completion Criteria: The system maintains response times under 3 seconds when supporting up to 1,000 concurrent users.

2.5 Supporting Tools

Unit Testing: Vitest (Frontend), Pytest (Backend).

Integration Testing: Postman, FastAPI TestClient.

System Testing: Selenium WebDriver (Python).

Performance Testing: Locust.

Quality Check: SonarQube (deployed via Docker)

3. Test Plan

3.1 Human Resources

Test Lead: Oversees all testing activities and ensures test coverage.

Test Engineers: Perform manual and automated testing (Selenium, Locust).

Developers: Handle unit testing (Vitest, Pytest) and bug fixes.

3.2 Test environment and tool

Quality Management (SonarQube): Analyzes source code quality automatically via Docker container to detect bugs and vulnerabilities.

Automation (Vitest, Pytest, Selenium, Postman): Executes automated test scripts at different levels.

Performance (Locust): Simulates user load and measures real-time API performance.

Environment & DevOps: Docker & Docker Compose (to orchestrate PostgreSQL, Redis, SonarQube, and Application containers in a Staging Environment).

4. Test Reports

4.1 Test Cases

Detailed records of Unit, Integration, System, Acceptance cases (e.g., stored in Report_Unit_Test.xls or automated HTML/XML reports generated by Pytest/Vitest).

4.2 Test Results Report

Provides detailed pass/fail results for each test case, defect statistics (extracted from Pytest/Vitest reports and SonarQube dashboard).

4.3 Final Evaluation

Summarizes the overall pass/fail rates, confirms whether testing completion criteria have been met, and provides recommendations based on SonarQube quality gates and Locust stress test results.

VI. Release Package & User Guides

1.Deliverable Package

No.	Deliverable Item	Description
1	Schedule/Task Tracking	We use JIRA to track our progress, Conference and to store schedules, notes and documents: • JIRA • Conference
2	Project Backlog	JIRA is a management tool that uses boards, lists, and cards to help organize our backlog
3	Source Codes	We host our source code for the Back End, Front End on Github: · Frontend: ReactJS + Vite + Ant Design/MUI. · Backend: Python FastAPI.
4	Database Script(s)	PostgreSQL.
5	Documents	A complete set of project documentation: SRS, SDD, Test Documentation, and Final Report.

6	User Guide	Manuals for system installation and usage instructions for all user roles.
7	Deployment Link	The URL to the live system deployed on Cloud platforms (Azure/AWS).

2.Installation Guides

Objective: Guide users to install CollabSphere on server or local environments.

2.1 Prerequisites

- Node.js >= 18
- Python >= 3.10
- PostgreSQL >= 13
- Docker & Docker Compose
- Cloud accounts (Azure, AWS, Cloudinary, Upstash Redis)

2.1.1 System Requirements

- Operating System: Windows 10/11, Linux, or macOS.
- Environment:
- Python 3.x (for Backend API).
- Node.js & npm (for ReactJS Frontend).
- PostgreSQL 13+ (Database management system).

2.2 Installation Instructions

2.2.1 Backend

- **Step 1:** Download & Install Visual Studio Code: [Visual Studio Code](#)

- **Step 2:** Download & Install Git: [Git](#)
- **Step 3:** Download & Install Docker: [Docker Desktop](#)
- **Step 4:** Download & Install Python: [Python 3.x](#)
- **Step 5:** Download & Install Postman: [Postman](#)
- **Step 6:** Navigate to the folder where you want to install the project, click on the address bar, type cmd, and press Enter.
- **Step 7:** Clone the repository using the command: git clone <backend-repo-url>.
- **Step 8:** Create a virtual environment and install dependencies: pip install -r requirements.txt.
- **Step 9:** Configure the environment variables by creating a .env file from the .env.example template.
- **Step 10:** Start the backend services and database using Docker or run python main.py.

2.2.2 Frontend

- **Step 1:** Download & Install Visual Studio Code: [Visual Studio Code](#)
- **Step 2:** Download & Install Git: [Git](#)
- **Step 3:** Download & Install Node.js: [Node.js v20+](#)
- **Step 4:** Download & Install Newman (optional for testing): [Newman](#)
- **Step 5:** Open your preferred terminal (CMD or PowerShell) and navigate to your workspace directory.
- **Step 6:** Clone the Frontend repository using the command: git clone <frontend-repo-url>.
- **Step 7:** Navigate into the project folder: cd cosre-frontend.

- **Step 8:** Install all necessary packages and dependencies by running the command:
npm install.
- **Step 9:** Create a .env.local file and update the VITE_API_BASE_URL to point to your backend.
- **Step 10:** Launch the application in development mode using the command: npm run dev.

3.User Manual

3.1 Roles and Features

- **Admin**
 - o View all user accounts
 - o Deactivate user accounts
 - o View system reports via email
 - o Access Dashboard
- **Staff**
 - o Import & manage subjects, syllabus, classes, and user accounts
 - o Assign lecturers & students to classes
- **Head Department**
 - o Approve/deny pending projects
 - o Assign projects to classes
 - o Update approved projects
- **Lecturer**
 - o Create/manage projects based on syllabus

- o Create/manage teams and team milestones
- o Monitor team progress & contributions
- o Submit pending projects for approval
- o Evaluate team members, milestones, and checkpoints
- o Use real-time collaboration tools (chat, video, whiteboard)

- **Student**

- o View assigned classes and teams
- o Manage team milestones and checkpoints
- o Monitor team progress and contributions
- o Submit work for evaluation
- o Chat and schedule meetings with teams
- o Use real-time collaboration tools (chat, video, whiteboard)

3.2 Workflow Examples

3.2.1 Login

- Step 1: Navigate to the CollabSphere login page. You will see the login interface with options to sign in using your credentials or Google account.

Choose Login Method

- Step 2: You have two options to log in:
Option A: Enter your email and password, then click Log in button
Option B: Click Continue with Google button to use your Google account

3.2.2 Access Dashboard

Upon successful login, you will be redirected to your dashboard where you can access all features based on your role (Student, Lecturer, Manager, or Admin).

3.2.3 Team Management

- Step 1: Navigate to Team Section from the sidebar menu, click on Team Management to access the team management interface.

- Step 2: Create Team Management

Look for the Create Team button in the team management interface and click it to open the team creation form.

3.2.4 Invite Member

- Step 1: Access Team Management

Navigate to Team Management from the sidebar to view your team details.

- Step 2: Click Invite Member Button

Look for the student icon with a plus sign (**+**) or the Invite Member button in the team interface.

- Step 3: Enter Member Email

In the invitation modal that appears:

Enter the email address of the student you want to invite

Optionally add a personal message

Click Send Invitation or Invite button

- Step 4: Wait for Response

The invited student will receive a notification and can accept or decline your invitation. You can track the invitation status in the team management interface.

3.2.5 Real-time Workspace (Main Screen)

The Real-time Workspace is the central hub for team collaboration, featuring multiple tools to help your team work together effectively on your capstone project.

3.2.6 Kanban Board Detail

- Step 1: Access Kanban Board

From the sidebar menu, navigate to Real-time Workspace → Kanban Board Detail

- Step 2: Add New Task

Click the + Add button in any column or use the Add New Task button to create a task:

- **Task Title:** Brief description (e.g., "Design Homepage Layout")
- **Description:** Detailed task information
- **Priority:** High, Medium, or Low
- **Due Date:** Deadline for the task
- **Assignee:** Team member responsible

- Step 3: Manage Tasks

Each task card displays:

- **Task title and description**
- **Priority badge** (color-coded)

- **Due date**
- **Edit icon** () - Modify task details
- **Delete icon** () - Remove task
- Step 4: Move Tasks Between Columns

Update task status by:

- **Drag and drop:** Click and drag task cards to different columns
- **Edit status:** Click edit icon and change status

3.2.7 Whiteboard Canvas

- Step 1: Open Whiteboard

Navigate to Real-time Workspace → Whiteboard Canvas to access the collaborative drawing board.

- Step 2: Drawing Tools

Use the toolbar to access drawing tools:

- **Pencil** (): Free-hand drawing
- **Text (T):** Add text boxes
- **Shapes:** Insert rectangles, circles, arrows
- **Sticky Notes:** Add colored notes
- **Eraser:** Remove elements
- **Color Picker:** Choose drawing colors
- Step 3: Navigation Controls

Control the canvas view:

- ← → **Arrows:** Undo/Redo actions
- **100% Zoom:** Adjust zoom level
- **+ Plus:** Add new elements
- **Pan:** Click and drag to move canvas
- Step 4: Share and Save

Manage your whiteboard:

- Click Share button to invite others
- Whiteboard auto-saves your work
- Export as image (if needed for documentation)

3.2.8 Video Meeting Room

- Step 1: Create Meeting

Navigate to Real-time Workspace → Video Meeting Room and click New Meeting button.

- Step 2: Join Meeting

Once a meeting is created:

- Meeting room opens automatically
- Meeting ID (abc-xyz) displayed at top
- Test your camera and microphone
- Click Join to enter the meeting

- Step 3: Meeting Controls

During the meeting, use these controls at the bottom:

-  **Microphone:** Mute/unmute audio
-  **Camera:** Turn video on/off
-  **Share Screen:** Share your screen
-  **Invite:** Invite more participants
-  **Chat:** Open meeting chat
-  **People:** View participant list
- **CC Captions:** Enable closed captions
-  **Settings:** Adjust audio/video setting
- Step 4: Participant Management

View and manage participants:

- Click People icon to see participant list
- View who's speaking (highlighted border)
- See who has camera/mic on or off
- Participant tiles show: You, Ngoc, An, Huy, Minh, Tai, etc.
- Step 5: Screen Sharing

To share your screen:

- Click Share Screen button
- Select which screen/window to share
- Choose "Entire screen" or specific application
- Click Share to start

- Click Stop Sharing when done
- Step 6: Leave Meeting

When you're ready to leave:

- Click the red Leave Call button
- Meeting continues for other participants
- You can rejoin anytime if meeting is still active

3.2.9 Submission Portal

- Step 1: Access Submission Portal

Navigate to Real-time Workspace → Submission Portal to submit your project milestones.

- Step 2: Choose Submission Method

You have two options for submission:

- **Files Upload:** Upload ZIP files directly
- **GitHub Link:** Submit via GitHub repository

- Step 3: GitHub Link Method

If submitting via GitHub:

- Switch to GitHub Link tab
- **Repository Link:** Paste your GitHub repo URL
- **Branch (Optional):** Specify branch name (default: main)
- **Commit Hash (Optional):** Specify exact commit

Step 4: Fill Submission Details

Complete the submission form:

- **Project Name:** Your project title
- **Description:** Brief description of this submission
- **Team Members:** Verify team member lists
- Step 5: Submit

After completing all fields and uploading files:

- Review all information for accuracy
- Click Submit button
- Confirm submission in the dialog
- You'll receive confirmation notification
- Mentor and reviewers will be notified

3.2.10 Peer Review Form

- Step 1: Access Peer Review Form

Navigate to Peer Review Form from the sidebar menu. This option becomes available after Review 3.

- Step 2: Rate Each Team Member

For each team member, provide:

- **Contribution Percentage:** Rate 0-100% based on actual work contribution
- **Attitude:** Professional behavior and cooperation
- **Communication:** Responsiveness and clarity

- **Quality of Work:** Technical skills and output quality
- **Detailed Comments:** Specific feedback and examples

Step 3: Self-Evaluation

As team leader, you must also evaluate your own contribution:

- Rate your own contribution percentage
- Reflect on your leadership effectiveness
- Identify areas for improvement
- Step 4: Submit Review

After completing all evaluations:

- Verify all ratings are completed
- Ensure total contribution percentages are reasonable
- Click Submit Review button
- Confirm submission (cannot be edited after submission)

3.2.11 Lecturer Dashboard

- Step 1: Access Lecturer Dashboard

After login, lecturers are directed to their personalized dashboard showing:

- Step 2: Dashboard Widgets

The dashboard displays key information:

- **My Teams:** List of teams you're mentoring
 - o Team name and members

- o Current project status
- o Upcoming deadlines
- **Pending Evaluations:** Ideas and topics awaiting your review
- **Review Schedule:** Your assigned review sessions
- **Council Assignments:** Topics assigned for council evaluation
- **Recent Activities:** Latest updates from your teams
- Step 3: Quick Actions

Access frequently used features directly:

- Create New Idea - Propose a new project topic
- Evaluate Ideas - Review pending student submissions
- View Schedule - Check review and defense schedules
- Provide Feedback - Submit mentor conclusions

3.2.12 Topic Management

- Step 1: Access Topic Library

Navigate to Topic Management to view and manage your proposed topics.

- Step 2: Create Multiple Topics

Lecturers can propose multiple project topics:

- **Solo Topics:** First 4 topics (can mentor alone)

- **Collaboration Topics:** Topic 5+ (require sub-mentor)
- Each topic includes: title, description, requirements, team size
- Topics go through council approval process
- Step 3: Manage Topic Requests

When students request your topics:

- View team details and member profiles
- Check if team size matches topic requirements
- Approve or reject requests with comments
- Assign sub-mentor if needed

3.2.13 Class Progress Monitoring

- Step 1: Access Class Monitoring

Navigate to Class Progress or Monitor Dashboard to view semester-wide statistics.

- Step 2: Key Metrics Displayed

The dashboard provides comprehensive overview:

- **Team Formation Status:**
 - o Total teams formed
 - o Students without teams
 - o Average team size
- **Idea Approval Progress:**
 - o Ideas pending mentor review

- o Ideas pending council review
- o Approved ideas count
- o Rejected/revision needed
- **Review Completion:**
 - o Review 1, 2, 3 completion rates
 - o Checklist submission status
 - o Teams needing attention
- **Defense Readiness:**
 - o Teams ready for Defense 1
 - o Teams ready for Defense 2
 - o Final results overview
- Step 3: Visual Analytics

View data through interactive charts:

- **Progress Timeline:** Gantt chart showing semester progression
- **Status Distribution:** Pie charts for team/idea statuses
- **Completion Rates:** Bar charts comparing review stages
- **Grade Distribution:** Histogram of defense results

Step 4: Update Topic Status

Track and update topic progression:

- **Draft** - Not submitted to council
- **Pending Council** - Awaiting council review

- **Approved** - Ready for team selection
- **Rejected** - Needs revision
- **Assigned** - Team working on it
- Step 5: Filter and Export

Customize your view and export data:

- Filter by mentor, specialty, or status
- Search for specific teams or students
- Export reports to Excel or PDF
- Schedule automated reports

3.2.14 Grading & Feedback Interface

- Step 1: Access Grading Interface

When evaluating ideas, topics, or defenses, you'll use the criteria-based grading form configured by the Manager for the semester.

- Step 2: Evaluate Each Criterion

Rate the project on multiple dimensions:

- **Innovation & Originality:** Uniqueness of the idea
- **Technical Feasibility:** Realistic implementation
- **Scope & Complexity:** Appropriate for capstone level
- **Documentation Quality:** Clarity and completeness
- **Presentation:** Communication and delivery
- Step 3: Provide Detailed Feedback

For each criterion, add specific comments:

- **Strengths:** What did the team do well?
- **Areas for Improvement:** Specific actionable suggestions
- **Examples:** Reference specific sections or aspects
- **Resources:** Recommend readings or tools if helpful
- Step 4: Submit Evaluation

After completing all ratings:

- Review all scores for consistency
- Ensure comments are constructive and professional
- Click Submit Evaluation
- System calculates overall score if applicable

3.2.15 Staff - Semester Management

Step 1: Navigate to Semester Management

From the admin panel, select Quản lý học kì (Semester Management) to view all semesters.

- Step 2: Click Create New Semester

Fill in the semester configuration form:

- **Semester Code:** e.g., "SP25" for Spring 2025
- **Semester Name:** Full name display
- **Prefix Name:** For generating topic codes
- **Start Date:** Semester begins

- **End Date:** Semester ends
- **Public Topic Date:** When topic list becomes visible
- Step 3: Set Lecturer Limits

Configure workload limits:

- **Solo Topic Limit:** Max topics lecturer can mentor alone (typically 4)
- **Sub-Mentor Limit:** Max topics as sub-mentor (typically 2-3)
- Step 4: Associate Criteria Form

Select the evaluation criteria form for this semester:

- Choose from existing criteria forms
- Or create a new custom form
- Different forms can be used for different evaluation stages
- Step 5: Save and Activate

Click Create Semester to save. The new semester will be available for:

- User registration and import
- Timeline configuration
- Role assignments for lecturers
- Student team formation

3.2.16 Staff - User Imports

Step 1: Download Student Template

Navigate to User Management → Import Students → Click Download Template

Step 2: Fill Student Data

Complete the Excel template with student information:

- **Student Code:** University ID (e.g., SE160585)
- **Full Name:** Last name and first name
- **Email:** Official university email (@fpt.edu.vn)
- **Phone:** Contact number
- **Specialty:** Software Engineering, AI, IoT, etc.
- **Date of Birth:** Format: DD/MM/YYYY

Step 3: Upload and Validate

Upload the completed file:

- Click Upload File and select your Excel
- System validates data format and checks for duplicates
- Preview shows data to be imported
- Review validation errors if any

Step 4: Confirm Import

After validation passes:

- Click Confirm Import
- System creates user accounts
- Sends welcome emails to all imported students

- View import summary (success/failed count)

3.2.17 Head of Dept - Statistics Dashboard

- Step 1: Access Statistics Dashboard

Navigate to Department Dashboard or Statistics to view comprehensive analytics.

- Step 2: Key Performance Indicators

View critical metrics at a glance:

- **Total Active Projects:** Current semester count
- **Completion Rate:** % of teams completing all milestones on time
- **Average Grade:** Overall defense performance
- **Faculty Workload:** Mentor/reviewer distribution
- **Topic Diversity:** Distribution across specialties

- Step 3: Trend Analysis

Compare performance across semesters:

- **Grade Trends:** Line chart showing average grades over time
- **Project Complexity:** Evolution of technical scope
- **Success Rates:** Completion vs dropout trends
- **Mentor Effectiveness:** Team performance by mentor

- Step 4: Specialty Analysis

Break down by department specialties:

- Project count per specialty

- Average performance by specialty
- Popular topic areas
- Resource allocation needs
- Step 5: Export Reports

Generate reports for stakeholders:

- **Executive Summary:** High-level overview PDF
- **Detailed Analytics:** Comprehensive Excel with all data
- **Comparative Reports:** Multi-semester analysis
- **Custom Reports:** Filter by date range, specialty, etc.

3.2.18 Head of Dept - Topic Bank Approval

- Step 1: Access Topic Bank

Navigate to Topic Bank → Pending Approval to view topics submitted by lecturers for inclusion in the official topic bank.

- Step 2: Evaluate Topic Quality

Review each topic based on department standards:

- **Academic Rigor:** Appropriate difficulty level
- **Relevance:** Aligns with curriculum and industry needs
- **Feasibility:** Can be completed within semester
- **Originality:** Not duplicate of existing topics
- **Learning Outcomes:** Clear educational value
- **Resources:** Available tools and support

- Step 3: Approve or Request Revision

Make decision on each topic:

- **Approve:** Add to official topic bank
- **Request Revision:** Send feedback to lecturer with specific improvements needed
- **Reject:** Decline with detailed explanation

- Step 4: Manage Topic Bank

Maintain the topic repository:

- Categorize topics by specialty and complexity
- Archive outdated or obsolete topics
- Update topic descriptions as technology evolves
- Track which topics are most popular
- Identify gaps in topic coverage

3.2.19 System Configuration

- Step 1: Access System Configuration

Navigate to Settings → System Configuration to access global system settings.

- Step 2: Configure Team Parameters

Set global team constraints:

- **Minimum Team Size:** Default 4 members
- **Maximum Team Size:** Default 6 members
- **Allow Solo Projects:** Enable/disable individual work
- **Team Formation Deadline:** Last date to form team

- Step 3: File Upload Settings

Configure file upload parameters:

- **Max File Size:** Set limit (e.g., 50MB)
- **Allowed File Types:** Whitelist extensions
- **Storage Quota:** Per-user storage limits
- **Scan for Viruses:** Enable/disable automatic scanning

- Step 4: Notification Settings

Configure notification behavior:

- **Email Notifications:** Enable/disable for different events
- **In-App Notifications:** Configure notification types
- **Notification Frequency:** Immediate or digest
- **Reminder Settings:** Days before deadline to send reminders